

VitaMate

تطبيق ذكي ومتكامل لتتبع الصحة ونمط الحياة

مقدم استكمالاً لمتطلبات الحصول على درجة الإجازة في هندسة البرمجيات ونظم المعلومات الذكية

إعداد:

سلام محمد العياش آمنة أمين زيتون

إشراف:

د. كادان الجمعة م. رغد الحسني

2026-2025

Syrian Arab Republic

Syrian Private University

Faculty of Artificial Intelligence Engineering

Department of Software Engineering and Intelligent Information Systems



VitaMate: An Intelligent Mobile Application for Integrated Health and Lifestyle Tracking

Senior Project 2 Report

Submitted in partial fulfillment of the requirements for the Bachelor's Degree in Software
Engineering and Intelligent Information Systems

Prepared by

Salam Mohammed Al-Ayash

Amenah Ayman Zaitoun

Supervised by

Dr. Kadan Jumaa

Eng. Raghad Al-Hossny

2025–2026

الإهداء:

إلى عائلتي، السند الأول والدائم، وإلى كل من أحاطني بالمحبة والدعم، وكان حضوره عونًا لي في مراحل هذا الطريق، أهدي ثمرة سنواتٍ من السعي والتعب، تقديرًا لكل كلمة تشجيع، وكل موقفٍ صادق، وكل دعاءٍ رافقني حتى وصلت إلى هذه اللحظة. وإلى أساتذتي ودكاترتي الأفاضل، الذين كان لعلمهم وتوجيههم أثرٌ كبير في مسيرتي الأكاديمية، أتقدم بخالص الامتنان والتقدير لما قدموه من معرفة وخبرة، ولما بذلوه من جهدٍ في سبيل أن نخطو بثبات نحو حياتنا المهنية.

وإلى مشرفينا الأعزاء، الدكتور كادان جمعة، والمهندسة رغد الحصني، والدكتور محمد البلخي، نتقدم بامتنانٍ خاص يليق بما قدموه لنا طوال رحلة المشروع؛ فلم يكن إشرافهم مجرد متابعة أكاديمية، بل كان توجيهًا حقيقيًا أسهم في صقل أفكارنا، وتصحيح مسارنا، ودفعنا في كل مرحلة إلى تقديم أفضل ما نستطيع. كان لخبرتهم، وملاحظاتهم الدقيقة، وثقتهم بقدرتنا على التطور أثرٌ واضح في الصورة التي وصل إليها هذا العمل اليوم.

نحمل لهم تقديرًا كبيرًا لما منحونا إياه من علمٍ ووقتٍ وخبرة، ولأنهم لم يكتفوا بأن يكونوا مشرفين على مشروع تخرج، بل كانوا جزءًا أساسيًا من رحلة تعلمٍ ستبقى آثارها معنا بعد انتهاء سنوات الدراسة.

إلى كل من كان جزءًا من هذه الرحلة، لكم من هذا النجاح نصيب، ولكم منا كل الشكر والامتنان.

Acknowledgements

We would like to express our sincere gratitude to our supervisors, Dr. Kadan Jumaa and Eng. Raghad Al-Hossny, for their guidance, constructive feedback, and continuous support throughout the development of the VitaMate project. Their academic and technical recommendations contributed significantly to refining the project and improving its overall quality.

We also extend our appreciation to the Faculty of Artificial Intelligence Engineering, the Department of Software Engineering and Intelligent Information Systems, and our families for their continuous support throughout our academic journey and the completion of this project.

Contents

Dedication	ii
Acknowledgements	iv
List of Tables	xii
List of Figures	xv
List of Abbreviations and Scientific Terms	xvi
الملخص	xvii
Abstract	xviii
Chapter 1: Introduction	1
1.1. Introduction	2
1.2. Project Overview and Background	2
1.3. Problem Statement and Its Practical and Academic Importance	2
1.4. Project Objectives	3
1.4.1. General Objective	3
1.4.2. Specific Objectives	3
1.5. Project Scope, Constraints, and Assumptions	4
1.5.1 In-Scope	4
1.5.2 Out-of-Scope	4
1.5.3 Project Constraints	4
1.5.4 Project Assumptions	5
1.6. Project Contribution	5
1.7. Research and Development Methodology	5
1.8. Report Organization	6
1.9. Key Definitions and Concepts	6

1.10. Chapter Summary	7
Chapter 2: Development Methodology and Project Management.....	8
2.1. Introduction.....	9
2.2 Senior Project 1 Baseline	9
2.3. Senior Project 2 Development Scope	10
2.4. Development Methodology – Agile Scrum	10
2.4.1 Agile Scrum	10
2.5. Project Schedule and Gantt Chart	11
2.6. Risk Management	12
2.7. Team Roles and Responsibilities	12
2.8. Quality Management and Continuous Monitoring	13
2.9. Project Management Tools and Software Configuration Management.....	13
2.10. Chapter Summary	14
Chapter 3: Theoretical Framework and Related Work	15
3.1. Introduction.....	16
3.2. Digital Health and Integrated Self-Monitoring.....	16
3.3. Behavior Change, Motivation, and Notifications	16
3.4. Personalized and Constraints-Aware Health Support.....	17
3.5. AI-Assisted Food Analysis	17
3.6. Relevant Technologies and Current Trends.....	18
3.7. Review of Related Studies	18
3.8. Comparative Analysis of Existing Applications.....	19
3.9. Identified Research and System Gap	20
3.10. Chapter Summary	20
Chapter 4: Requirements Analysis and System Modeling	21

4.1 Introduction.....	22
4.2 Feasibility Study	22
4.3 Requirements Elicitation and Sources	22
4.4 Stakeholders and Needs Analysis	23
4.5 Functional Requirements	23
4.6 Non-Functional Requirements	26
4.7 Requirements Prioritization Using MoSCoW	27
4.8 UML Requirements Models	28
4.8.1 Main Use Cases.....	28
4.8.2 Overall Use Case Diagram.....	28
4.8.3 Summary of Main Use Case Specifications.....	29
4.8.4 Selected Activity Diagrams	30
4.8.5 Selected Sequence Diagrams	33
4.9 Agile Requirements Models	34
4.9.1 User Stories	34
4.9.2 Story Mapping	35
4.9.3 Product Backlog.....	35
4.9.4 Sprint Backlog	36
4.10 Requirements Traceability and Initial Test Cases	36
4.10.1 Requirements Traceability Matrix	37
4.10.2 Initial Test Cases.....	37
4.11 Chapter Summary	38
Chapter 5: Architectural and Detailed Design	39
5.1 Introduction.....	40
5.2 High-Level Architecture and Architectural Pattern.....	40

5.3 Data Design.....	41
5.4 API and Security Design.....	43
5.4.1 API Structure and Conventions	43
5.4.2 Authentication, Authorization, and Data Protection.....	44
5.4.3 AI Food-Analysis Interface.....	44
5.5 Senior Project 2 Component Design.....	44
5.5.1 Motivation System Design.....	45
5.5.2 Notification System Design	45
5.5.3 UI/UX Design	46
5.5.4 AI Food-Analysis Design	46
5.6 Cross-Module Coordination.....	47
5.7 Chapter Summary	47
Chapter 6: Development Environment and Software Implementation.....	48
6.1 Introduction.....	49
6.2 Senior Project 1 Implementation Baseline.....	49
6.3 Development Environment and Engineering Setup.....	49
6.4 Senior Project 2 Implementation	50
6.4.1 Motivation System Implementation.....	50
6.4.2 Notification System Implementation	51
6.4.3 UI/UX Refinement.....	52
6.4.4 AI Food-Analysis Service Implementation	54
6.4.5 Cross-Module Integration and Stabilization	55
6.5 Implementation Challenges and Solutions.....	56
6.6 Chapter Summary	56
Chapter 7: Testing and Evaluation.....	58

7.1 Introduction and Test Strategy.....	59
7.2 Test Environment and Scope	60
7.3 Representative Test Cases and Backend Results	60
7.4 Frontend and UI/UX Validation	61
7.5 Integration, Security, and Privacy.....	61
7.6 Performance and AI Evaluation.....	62
7.7 Acceptance Status and Remaining Validation.....	63
7.8 Chapter Summary	64
Chapter 8: Results and Analysis	65
8.1 Final System Outcomes	66
8.2 Senior Project 2 Results	66
8.3 Comparative Analysis with Existing Systems	67
8.4 Achievement of Project Objectives	67
8.5 Key Performance Indicators and AI Results.....	68
8.6 Strengths and Limitations	69
8.7 Chapter Summary	70
Conclusion	71
Main Achievements	71
Challenges and Lessons Learned.....	71
Recommendations and Future Work	72
References.....	73
Appendices.....	77
Appendix A: Detailed Use Case Specifications.....	77
UC-01 — Manage Account and Health Profile	77
UC-02 — Manage Daily Health Tracking.....	77

UC-03 — Manage Nutrition and AI-Assisted Meals	78
UC-04 — Manage Medications and Chronic Conditions.....	78
UC-05 — Manage Goals and Unhealthy Habits.....	79
UC-06 — Manage Notifications and Motivational Support.....	79
UC-07 — View Reports and Supportive Guidance	79
Appendix B: Additional UML and Agile Artifacts	81
B.1 Additional Activity Diagram.....	81
B.2 Additional Sequence Diagram	82
B.3 Notification Planning and Delivery Sequence Diagram	83
B.4 Detailed Agile Backlog	83
Appendix C: Detailed Test Plan, Traceability, and Evidence	85
C.1 Requirement-Level Traceability	85
C.2 Executed Representative Test Cases	91
C.3 Evidence Summary	91
Appendix D: User Manual	93
D.1 Overview and Operating Requirements.....	93
D.2 Installation and Setup.....	93
D.3 Quick Start	93
D.4 Main Interfaces	94
D.5 Notifications and Motivation	95
D.6 AI-Assisted Meal Logging.....	96
D.7 Troubleshooting	96
D.8 Frequently Asked Questions (FAQ)	96
D.9 Support and Contact.....	97
Appendix E: Technical Evidence and Representative Code.....	98

E.1 Simplified Project Structure	98
E.2 Configuration and CI Evidence	98
E.3 Representative Data-Access and Business Logic	99
E.4 Representative API Implementation.....	100
E.5 Representative Relational Schema	101
E.6 Representative API Catalogue.....	102
Appendix F: Research Ethics and Use of Artificial Intelligence Tools.....	103

List of Tables

Table 1: Project Constraints.....	4
Table 2: Completed Senior Project 1 Outcomes.....	9
Table 3: Main Development Areas of Senior Project 2.....	10
Table 4: Scrum Activities and Definition of Done.....	11
Table 5: Senior Project 2 Risk Register.....	12
Table 6: Team Roles and Responsibilities.....	12
Table 7: Quality-Control Mechanisms.....	13
Table 8: Project Management and Collaboration Tools.....	13
Table 9: Current Digital-Health Trends Relevant to VitaMate.....	18
Table 10: Summary of Related Studies.....	18
Table 11: Comparison of Existing Health Applications.....	19
Table 12: VitaMate Feasibility Assessment.....	22
Table 13: Requirements Sources.....	22
Table 14: Stakeholders and Main Needs.....	23
Table 15: Consolidated Functional Requirements of the VitaMate System.....	23
Table 16: Approved SRS Functional-Domain Implementation Status.....	26
Table 17: Non-Functional Requirements of the VitaMate System.....	26
Table 18: MoSCoW Prioritization of VitaMate Requirements.....	27
Table 19: Main Use Cases of VitaMate.....	28
Table 20: Summary of Main Use Case Specifications.....	29
Table 21: Representative VitaMate User Stories.....	34
Table 22: VitaMate User Story Map.....	35
Table 23: Senior Project 2 Product Backlog Summary.....	35
Table 24: VitaMate Sprint Backlog Summary.....	36
Table 25: VitaMate Requirements Traceability Summary.....	37
Table 26: Representative Initial Test Cases.....	37
Table 27: Main Database Domains and Responsibilities.....	43
Table 28: General API Design Conventions.....	43
Table 29: Main VitaMate API Groups.....	43

Table 30: Representative AI-Meal API Contract.....	44
Table 31: Motivation-System Design Responsibilities.....	45
Table 32: Notification-System Design Responsibilities.....	45
Table 33: UI/UX Design Principles Applied in Senior Project 2	46
Table 34: AI Food-Analysis Design Responsibilities.....	46
Table 35: Main Development Tools and Verified Versions.....	49
Table 36: Engineering Environment Summary	50
Table 37: Motivation Implementation Flow	50
Table 38: Notification Planning and Delivery Flow	51
Table 39: Notification-Flow Responsibilities.....	51
Table 40: Main AI Pipeline Components	55
Table 41: Main Senior Project 2 Additions and Refinements	56
Table 42: Main Implementation Challenges and Applied Solutions.....	56
Table 43: Test Strategy Summary.....	59
Table 44: Coverage of Testing Types Required by the Project Guide	59
Table 45: Included Test Scope.....	60
Table 46: Representative Test Cases and Results.....	60
Table 47: Backend Test Summary	61
Table 48: Frontend Test Summary	61
Table 49: Integration and Security Status.....	61
Table 50: Targeted Backend Performance Optimization Evidence.....	62
Table 51: Verified AI Performance Improvement.....	62
Table 52: Verified Quality and AI Metrics.....	62
Table 53: Acceptance Status	63
Table 54: Final Implemented System Capabilities	66
Table 55: Main Senior Project 2 Results	66
Table 56: Comparative Analysis of VitaMate and Existing Applications.....	67
Table 57: Achievement of Project Objectives	67
Table 58: Final Key Performance Indicators	68
Table 59: Main Strengths and Limitations.....	69
Table 60: Detailed Use Case Specification - UC-01: Manage Account and Health Profile...	77

Table 61: Detailed Use Case Specification - UC-02: Manage Daily Health Tracking	77
Table 62: Detailed Use Case Specification - UC-03: Manage Nutrition and AI-Assisted Meals	78
Table 63: Detailed Use Case Specification - UC-04: Manage Medications and Chronic Conditions	78
Table 64: Detailed Use Case Specification - UC-05: Manage Goals and Unhealthy Habits ..	79
Table 65: Detailed Use Case Specification - UC-06: Manage Notifications and Motivational Support	79
Table 66: Detailed Use Case Specification - UC-07: View Reports and Supportive Guidance	79
Table 67: Detailed Senior Project 2 Backlog and Acceptance Focus	83
Table 68: Final Sprint Objectives and Deliverables	83
Table 69: Final SRS-Level Requirements Traceability Matrix	85
Table 70: Detailed Representative Executed Tests	91
Table 71: Final Test Evidence Map	91
Table 72: Common User Issues and Responses	96
Table 73: Key Configuration Files	98
Table 74: Main CI Checks	98
Table 75: Representative Logical Relational Schema	101
Table 76: Representative API Interface Catalogue	102
Table 77: AI-Assistance Disclosure	103

List of Figures

Figure 1: Gantt Chart for Senior Project 2.....	11
Figure 2: High-Level Use Case Diagram of VitaMate	29
Figure 3: Activity Diagram for AI-Assisted Meal Analysis.....	31
Figure 4: Activity Diagram for Health-Tracker Recording and Motivation-State Update.....	32
Figure 5: Sequence Diagram for AI-Assisted Meal Analysis.....	33
Figure 6: Sequence Diagram for Health-Tracker and Motivation Update	34
Figure 7: Final High-Level Architecture of VitaMate	41
Figure 8: Entity–Relationship Diagram of VitaMate.....	42
Figure 9: VitaMate Home and Daily Health Overview	53
Figure 10: Activity and Steps Interface	53
Figure 11: Nutrition Tracking Interface.....	54
Figure 12: Profile and Self-Management Interface	54
Figure 13: AI Food-Analysis Flow	55
Figure 14: AI Food-Analysis Average Runtime: Baseline vs Final Pipeline	69
Figure 15: Activity Diagram for Unhealthy-Habit Cessation.....	81
Figure 16: Sequence Diagram for Unhealthy-Habit Cessation and Motivation Update	82
Figure 17: Sequence Diagram for Notification Planning, Synchronization, and Local Delivery	83
Figure 18: VitaMate Login Interface	94
Figure 19: Hydration Tracking Interface	94
Figure 20: Unhealthy-Habit Management Interface.....	95
Figure 21: Medication Management Interface.....	95
Figure 22: Representative Django ORM and Data-Access Code.....	99
Figure 23: Representative Backend Business-Logic Code.....	99
Figure 24: Representative AI-Meal REST Endpoint Implementation.....	100

List of Abbreviations and Scientific Terms

Abbreviation	Full Term
AI	Artificial Intelligence
API	Application Programming Interface
CI/CD	Continuous Integration and Continuous Deployment
DRF	Django REST Framework
ERD	Entity–Relationship Diagram
JWT	JSON Web Token
ML	Machine Learning
REST	Representational State Transfer
RTM	Requirements Traceability Matrix
SRS	Software Requirements Specification
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

الملخص

(VitaMate) هو تطبيق جوال ذكي طُوّر لمعالجة مشكلة تشتت خدمات الصحة الرقمية وتتبع نمط الحياة، وذلك من خلال دمج عدة مجالات صحية يومية ضمن منصة واحدة مترابطة. هدف المشروع إلى دعم المتابعة الذاتية الآمنة للتغذية والترطيب والنشاط البدني والنوم والأدوية والحالات المزمنة والأهداف الشخصية والعادات غير الصحية والمغذيات الدقيقة والتقدم والتنبيهات والتحفيز، مع الحفاظ على حدود واضحة تمنع تقديم التطبيق على أنه أداة للتشخيص أو العلاج.

طُوّر المشروع تدريجياً بالاعتماد على منهجية Agile Scrum. أسس مشروع التخرج الأول المتطلبات والمعمارية والبنية الأساسية لتطبيق Flutter وخدمات Django وقاعدة بيانات PostgreSQL والمتتبعات الرئيسية. أما مشروع التخرج الثاني فركز على أربعة محاور رئيسية: تطوير نظام تحفيز مرتبط بأفعال المستخدم الموثقة، وتطوير مسار إشعارات منسق بين الخادم والجهاز، وتحسين شامل لتجربة وواجهات المستخدم، وإعادة تطوير ودمج خدمة تحليل صور الطعام بالذكاء الاصطناعي. اعتمدت البنية النهائية على تطبيق Flutter لنظام Android، وخلفية Django REST، وقاعدة بيانات PostgreSQL، وخدمة FastAPI مستقلة لتحليل صور الوجبات. بقيت نتائج الذكاء الاصطناعي مؤقتة إلى أن يراجعها المستخدم ويؤكد لها، ثم حُسبت القيم الغذائية النهائية من قاعدة التغذية المرجعية الخاصة بالتطبيق.

شمل التقييم اختبارات للخلفية وFlutter والتكامل والخصوصية والانحدار والواجهات والأداء والذكاء الاصطناعي. نجحت 240 حالة اختبار للخلفية، واجتازت اختبارات Flutter جميع الحالات مع تغطية أسطر بلغت 53.26%، ونجح المسار الرئيسي بين التطبيق والخلفية. كما أكملت خدمة تحليل الطعام 35 من 35 صورة دون أخطاء تشغيل، وانخفض متوسط زمن التحليل من 123.40 ثانية إلى 6.81 ثانية، وبلغ متوسط IoU للتقسيم 90.74%، ودقة Top-3 للتعرف على الطبق 57.14%، ونجاح ربط النتائج النهائية 91.43%.

أظهر النظام النهائي حلاً برمجياً متكاملاً وقابلاً للاختبار لدعم الإدارة الذاتية لنمط الحياة. أما خدمة الذكاء الاصطناعي الثانية الخاصة بالتوصيات الغذائية المخصصة فلم تُنفذ ضمن النسخة المقدمة، ولذلك وُثقت كعمل مستقبلي ولم تُعرض كإنجاز مكتمل. الكلمات المفتاحية: الصحة الرقمية، الصحة المتنقلة، تتبع الصحي المتكامل، نظام التحفيز، نظام الإشعارات، تحليل الطعام بالذكاء الاصطناعي، هندسة البرمجيات.

Abstract

VitaMate is an intelligent mobile application developed to address fragmentation in digital health and lifestyle tracking by integrating several daily health domains within one coordinated platform. The project aimed to support secure self-monitoring of nutrition, hydration, physical activity, sleep, medications, chronic conditions, personal goals, unhealthy habits, micronutrients, progress, reminders, and motivational feedback while maintaining clear non-diagnostic boundaries.

The project was developed incrementally using Agile Scrum. Senior Project 1 established the requirements, architecture, Flutter and Django foundations, PostgreSQL data layer, and core tracking modules. Senior Project 2 focused on four principal developments: a motivation system linked to verified user actions, a coordinated backend-to-device notification workflow, broad UI/UX refinement, and redevelopment and integration of the AI food-analysis service. The final architecture used a Flutter Android client, a Django REST backend, PostgreSQL, and a separate FastAPI runtime for meal-image analysis. AI results remained provisional until user review and confirmation, after which final nutrition values were calculated from the application's canonical nutrition catalog.

Evaluation combined backend, Flutter, integration, privacy, regression, interface, performance, and AI-related evidence. The backend suite completed 240 tests successfully, all Flutter tests passed with 53.26% line coverage, and the main mobile-to-backend flow passed. The final AI pipeline completed 35/35 release images with no runtime failures, reduced average analysis time from 123.40 seconds to 6.81 seconds, achieved 90.74% mean segmentation IoU, 57.14% dish Top-3 accuracy, and 91.43% finalization-mapping success.

The final system demonstrated an integrated and testable software-engineering solution for lifestyle self-management. The second AI service for personalized dietary recommendations was not implemented in the submitted release and was retained as future work rather than reported as a completed result.

Keywords: Digital Health, Mobile Health, Integrated Health Tracking, Motivation System, Notification System, AI Food Analysis, Software Engineering.

Chapter 1:

Introduction

1.1. Introduction

VitaMate is an intelligent mobile application designed to integrate multiple health and lifestyle-tracking services within a unified platform. It supports nutrition, hydration, physical activity, sleep, medication management, chronic-condition monitoring, personal goals, unhealthy habits, notifications, motivational feedback, and AI-assisted food analysis.

The project was developed across two academic stages. Senior Project 1 established the main requirements, architecture, backend and frontend foundations, and core tracking modules. Senior Project 2 focused on system integration and refinement, with particular emphasis on the motivation system, notification workflow, user-interface usability, AI food-analysis service, and final testing and evaluation.

1.2. Project Overview and Background

Digital-health applications support users in recording daily health information, reviewing progress, receiving reminders, and managing lifestyle-related behaviors. However, many applications focus on individual areas such as nutrition, activity, hydration, sleep, or medication management.

These health dimensions are often interrelated. Chronic conditions, personal goals, medications, and daily behaviors may affect nutrition targets, hydration, activity, sleep, reminders, and other tracking decisions. Managing these areas independently can therefore result in fragmented information and inconsistent user guidance.

VitaMate addresses this fragmentation through an integrated mobile platform that combines major health and lifestyle domains and coordinates them through shared user data and backend services. The system is intended for self-monitoring and lifestyle support and does not provide medical diagnosis or replace professional healthcare advice.

1.3. Problem Statement and Its Practical and Academic Importance

Many health and lifestyle applications manage nutrition, hydration, activity, sleep, medications, habits, and health conditions as separate functions. This fragmentation increases user effort and

limits the ability of the system to coordinate related health information, goals, restrictions, reminders, and progress feedback.

The problem becomes more important when several domains affect one another. For example, chronic conditions and medications may influence nutrition, hydration, physical activity, and personal targets. A system that treats these domains independently may therefore provide incomplete or inconsistent support.

From a practical perspective, VitaMate aims to reduce this fragmentation by providing a unified environment for daily tracking, reminders, progress monitoring, and motivational feedback. From a software-engineering perspective, the project demonstrates the design and implementation of a multi-domain health platform with centralized coordination, modular services, intelligent assistance, and structured testing.

1.4. Project Objectives

1.4.1. General Objective

The general objective of VitaMate is to design, implement, and evaluate an integrated mobile application that supports health and lifestyle self-monitoring through coordinated tracking, personalized constraints, reminders, motivational feedback, progress analysis, and selected intelligent services.

1.4.2. Specific Objectives

1. Develop secure user-account and health-profile management for personalized tracking.
2. Integrate nutrition, hydration, activity, sleep, medication, chronic-condition, goal, and habit-management functions within one mobile platform.
3. Coordinate health-related targets, restrictions, and tracker behavior through a centralized constraints mechanism.
4. Provide progress dashboards, reports, and cross-domain health summaries based on recorded user data.
5. Develop a motivation system that links verified user actions with points, missions, streaks, achievements, and motivational feedback.

6. Develop a coordinated notification system that supports user preferences, reminder planning, synchronization, and local device delivery.
7. Provide review-assisted AI food-image analysis integrated with the nutrition workflow.
8. Improve system usability and evaluate the final implementation through functional, integration, security, performance, interface, and AI-related testing.

1.5. Project Scope, Constraints, and Assumptions

1.5.1 In-Scope

VitaMate includes user authentication and health profiles; nutrition, hydration, activity, steps, sleep, medication, chronic-condition, goal, and unhealthy-habit tracking; coordinated health constraints; motivational features; notifications; dashboards and progress reporting; and review-assisted AI food analysis. The mobile application communicates with the Django backend and PostgreSQL database through REST APIs.

1.5.2 Out-of-Scope

VitaMate is not a diagnostic, treatment, hospital-management, or electronic-medical-record system. It does not independently modify medication or treatment plans. Integration with all wearable or medical devices is outside the current scope. AI-generated food information is treated as an assistive estimate and requires user review before final nutrition records are created.

The approved SRS also specified a second AI service for personalized dietary recommendations (SRS FR-60-FR-66). This service was not implemented in the submitted release and is therefore treated as deferred future work rather than part of the final implemented scope.

1.5.3 Project Constraints

Table 1: Project Constraints

Constraint	Description
Time	Development and evaluation are limited by the academic schedule.
Team Size	The project is developed by two students.
Technical Dependencies	The application depends on backend, database, libraries, and Android services.

Constraint	Description
AI Data	AI performance depends on available datasets and supported food classes.
Health Safety	Health-related guidance must remain cautious and non-diagnostic.
Privacy	Personal health data requires authenticated and controlled access.

1.5.4 Project Assumptions

1. Required development and testing environments remain available.
2. Frontend and backend components can communicate through defined API contracts.
3. Target Android devices provide the required notification and activity permissions.
4. Academic feedback is available during development and evaluation.

1.6. Project Contribution

VitaMate’s main contribution is architectural and integrative rather than diagnostic. The system combines multiple health and lifestyle domains within one platform and coordinates shared goals, chronic-condition restrictions, and tracker behavior through a centralized Constraints Layer.

The project also extends this integrated architecture through a motivation system, a coordinated backend-to-device notification workflow, improved user-interface usability, and review-assisted AI food analysis. These components are designed to work with the existing tracking modules rather than as isolated features.

From a software-engineering perspective, VitaMate demonstrates the development of a modular mobile health platform that combines requirements engineering, layered backend services, mobile development, intelligent-service integration, cross-module coordination, and structured testing.

1.7. Research and Development Methodology

VitaMate followed an applied software-engineering approach combining descriptive and analytical review with engineering implementation and evaluation. Related studies and existing applications were reviewed to identify the project gap and guide requirements and design decisions.

The system was developed iteratively using Agile Scrum. Requirements, design, implementation, integration, and testing were refined across successive sprints. Senior Project 1 established the project baseline, while Senior Project 2 concentrated on integration, motivation and notification development, interface refinement, AI-service improvement, and final system evaluation.

1.8. Report Organization

This report consists of eight chapters. Chapter 1 introduces the project, while Chapter 2 presents the development methodology and project management. Chapter 3 reviews the theoretical framework and related work. Chapter 4 presents requirements and system modeling, and Chapter 5 describes the architectural and detailed design. Chapter 6 focuses on the implementation, particularly the major Senior Project 2 developments. Chapter 7 presents testing and evaluation, while Chapter 8 analyzes the final results and achievement of project objectives. The report concludes with the main outcomes, limitations, recommendations, references, and supporting appendices.

1.9. Key Definitions and Concepts

Integrated Health Tracking: Coordinated management of multiple health and lifestyle domains through shared data and system services.

Constraints Layer: A centralized rule-based backend component that resolves applicable health restrictions, goals, and tracker constraints.

Health Tracker Coordinator: A backend coordination component that assembles cross-domain health information for unified views and progress summaries.

AI-Assisted Food Analysis: Computer-vision-based meal analysis that produces provisional food information for user review before final nutrition records are created.

Motivation System: The VitaMate component that manages points, missions, streaks, achievements, and motivational feedback based on verified user actions.

1.10. Chapter Summary

This chapter introduced VitaMate, defined the problem, objectives, scope, and main contribution, and summarized the methodology and report structure. The following chapter presents the development methodology and project-management approach used throughout the project.

Chapter 2: Development Methodology and Project Management

2.1. Introduction

VitaMate is a multi-module software project that combines a mobile application, backend services, a centralized database, health-tracking modules, notifications, and intelligent services. Its interconnected components require iterative development, continuous integration, and systematic quality control.

The project was developed across two academic stages. Senior Project 1 established the requirements, planning, preliminary design, implementation foundations, and an initial AI food-analysis prototype. Senior Project 2 built on this baseline with particular emphasis on developing the motivation and notification systems, refining the user interface and user experience, integrating and improving the AI food-analysis service, and completing the required integration, testing, and final evaluation.

2.2 Senior Project 1 Baseline

Senior Project 1 transformed the initial VitaMate concept into a structured software-engineering baseline. It established the main requirements and project plan, preliminary architecture and database design, Flutter and Django REST foundations, an independent AI food-analysis prototype, and initial quality-assurance mechanisms. Work that remained incomplete, non-integrated, or insufficiently validated was transferred to Senior Project 2 rather than being considered a completed outcome.

Table 2: Completed Senior Project 1 Outcomes

Area	Completed Senior Project 1 Baseline
Requirements & Analysis	Problem, objectives, scope, stakeholders, requirements, use cases, initial RTM and test cases
Planning	Project plan, backlog, sprints, Gantt chart, responsibilities and risks
Design	Architecture, database design, UML, API planning and initial UI/UX
Backend	Django REST foundation, database connectivity, authentication and reusable services
Frontend	Flutter structure, routing, API client and reusable UI components

Area	Completed Senior Project 1 Baseline
AI	Standalone experimental food-analysis prototype
Quality Assurance	Initial backend, Flutter, integration, performance and CI-related checks

2.3. Senior Project 2 Development Scope

Senior Project 2 focused on transforming the existing VitaMate baseline into a more integrated, usable, and technically stable final system. The main development work was concentrated in four areas:

Table 3: Main Development Areas of Senior Project 2

Development Area	Main Focus
Motivation System	Developing and integrating motivational feedback and progress mechanisms with user actions and tracking modules
Notification System	Developing a coordinated notification workflow and integrating reminders with relevant application services
UI/UX Refinement	Redesigning and refining application interfaces to improve clarity, consistency, navigation, and ease of use
AI Food-Analysis Service	Improving the existing AI prototype and integrating it with the backend, nutrition workflow, and user-review process

Supporting activities included integration stabilization, defect correction, regression and integration testing, documentation refinement, and final system evaluation. The implementation and verified results of these activities are presented in Chapters 6–8.

2.4. Development Methodology – Agile Scrum

2.4.1 Agile Scrum

Agile Scrum was adopted as the primary development methodology because VitaMate consists of several interconnected modules that require incremental implementation, frequent integration, testing, and refinement. Scrum allowed technical issues and changing priorities to be addressed during development instead of postponing integration until the end of the project.

The methodology was adapted to the two-member project team through lightweight sprint planning. Each sprint defined a clear objective and selected backlog items, followed by implementation, integration, testing, defect correction, and review. Unfinished or newly identified work was returned to the Product Backlog and reprioritized for subsequent sprints.

Table 4: Scrum Activities and Definition of Done

Sprint Stage	Application in VitaMate
Planning	Select priority backlog items and define the sprint objective
Development	Implement frontend, backend, AI, or shared-service changes
Integration	Connect affected components and verify cross-module behavior
Testing	Execute relevant functional, integration and regression tests
Review	Evaluate completed work and return unresolved items to the backlog

A backlog item was considered complete only after its required implementation, integration, validation, testing, and documentation had been completed.

2.5. Project Schedule and Gantt Chart

The Senior Project 2 schedule was organized into iterative development and stabilization periods. The main activities included project-scope refinement, motivation-system development, notification-system development, UI/UX refinement, AI food-analysis integration, system integration, testing, documentation, and final delivery.

PROJECT PLANNING & GANTT CHART

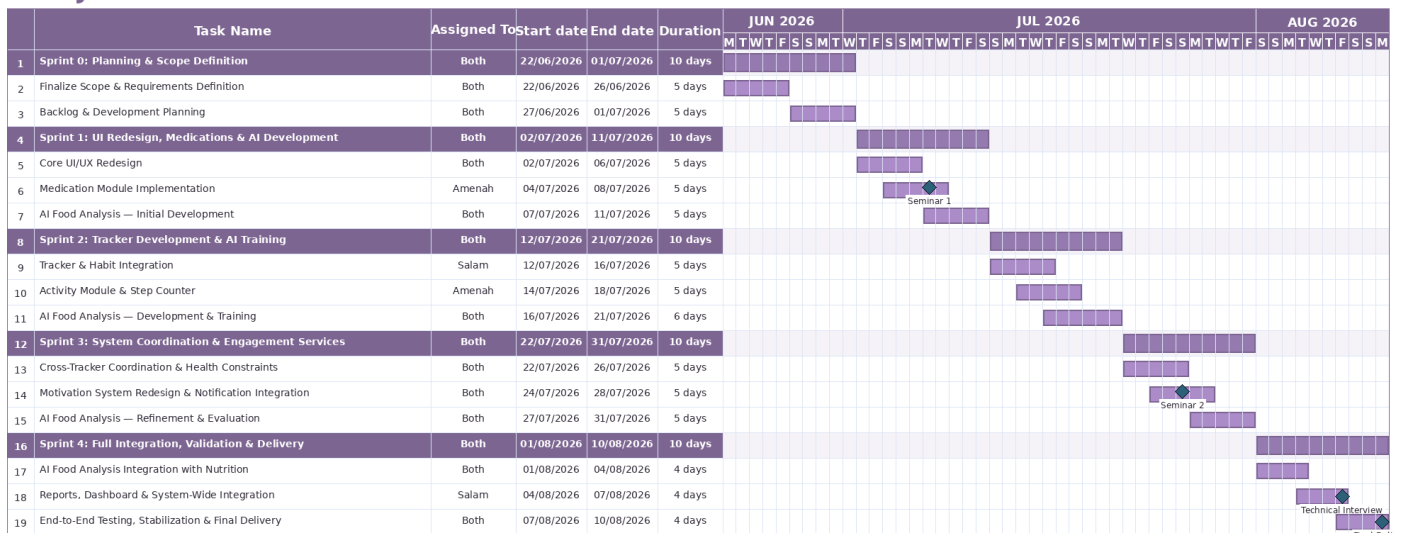


Figure 1: Gantt Chart for Senior Project 2

2.6. Risk Management

Risk management is treated as a continuous activity integrated with sprint planning, backlog refinement, testing, and project reviews. Risks are evaluated according to their probability and potential impact, and each major risk is associated with a preventive or corrective response.

Table 5: Senior Project 2 Risk Register

Risk	Probability	Impact	Main Response
Schedule pressure	High	High	Prioritize essential work and reserve time for testing
Cross-module integration defects	High	High	Incremental integration and regression testing
AI reliability limitations	High	High	User review, cautious output and manual correction
Notification/device-permission failures	Medium	Medium	Validate permissions and scheduling behavior
Health-rule inconsistency	Medium	High	Centralize shared constraints and verify conflicting cases
Security/privacy defects	Medium	High	Authentication, access control, secret protection and testing

2.7. Team Roles and Responsibilities

The project team consists of two students supported by two academic supervisors. Responsibilities were distributed according to the technical needs of each sprint, while integration, testing, review, and documentation remained shared responsibilities.

Table 6: Team Roles and Responsibilities

Member	Role	Main Responsibilities
Dr. Kadan Jumaa	Academic Supervisor	Academic guidance, project-direction review, quality follow-up, and evaluation of major deliverables
Eng. Raghad Al-Hossny	Technical Supervisor	Technical and organizational guidance, progress review, and feedback on development stages
Amenah Ayman Zaitoun	Software Engineer	Requirements analysis, documentation, system design, technical coordination, and contribution to frontend and backend development
Salam Mohammed Al-Ayash	Software Engineer	Frontend and backend development, database integration, implementation of system functions, testing, and technical refinement

Member	Role	Main Responsibilities
Both Students	Shared Project Team	Sprint planning, integration, code review, testing, documentation, issue resolution, and presentation preparation

2.8. Quality Management and Continuous Monitoring

Quality was managed continuously throughout development rather than being evaluated only after implementation. Major changes were reviewed, integrated incrementally, and verified through automated and manual testing where applicable. Defects and incomplete tasks were returned to the backlog for correction.

Project progress was monitored through sprint objectives, integration results, test evidence, risk reviews, supervisor feedback, and comparison with the planned schedule.

Table 7: Quality-Control Mechanisms

Quality Mechanism	Purpose
Acceptance Criteria	Defines when work is considered complete
Code Review	Detects logic and maintainability issues
Automated Testing	Verifies repeatable behavior
Integration & Regression Testing	Detects cross-module and newly introduced defects
Security Checks	Verifies access control and sensitive configuration
Traceability Review	Links requirements with implementation and testing
Documentation Review	Keeps documentation aligned with the implemented system

2.9. Project Management Tools and Software Configuration Management

Project management relied on lightweight tools suitable for the size of the team. The emphasis was placed on clear planning records and verifiable technical outputs rather than complex administrative processes.

Table 8: Project Management and Collaboration Tools

Tool / Artifact	Purpose
Product Backlog	Requirements, improvements, defects and development tasks
Sprint Plan	Sprint objectives and workload
Gantt Chart	Timeline and milestones
RTM	Requirement-to-design/test traceability
Git Repository	Version control and collaboration
CI/Test Reports	Automated verification and technical evidence

Git was used for source-code version control and change coordination. Development changes were reviewed and tested before integration, while dependency files, database migrations, environment configuration, and technical documentation were kept synchronized with the implementation. Credentials and sensitive configuration were excluded from the repository.

2.10. Chapter Summary

This chapter summarized the Senior Project 1 baseline and defined the principal Senior Project 2 development scope. It also presented the Agile Scrum approach, project schedule, risk and quality management, team responsibilities, monitoring practices, and software configuration management used to guide the final development stage. The following chapter presents the theoretical framework and related work supporting the VitaMate project.

Chapter 3: Theoretical Framework and Related Work

3.1. Introduction

This chapter presents the theoretical and research foundations relevant to VitaMate, including integrated digital-health tracking, behavior-change and motivational support, personalized coordination, and AI-assisted food analysis. It also reviews related studies and existing health applications to identify the system gap addressed by VitaMate and summarizes the technologies and current trends relevant to its design.

3.2. Digital Health and Integrated Self-Monitoring

Digital health encompasses the use of software, mobile technologies, connected devices, health data, and intelligent techniques to support health-related needs. Mobile health applications provide an accessible environment for recording daily data, reviewing progress, receiving reminders, and supporting personal self-management [1].

Digital self-monitoring can support the recording and review of behaviors such as nutrition, activity, hydration, sleep, medications, and personal goals. Research has shown that digital self-monitoring can contribute to improvements in lifestyle-related outcomes, particularly when interventions are tailored to the individual [2].

When these domains are managed through separate applications, however, information and feedback may become fragmented. VitaMate therefore adopts an integrated approach in which several tracking domains use shared user information and contribute to coordinated progress summaries, reminders, constraints, and supportive feedback.

3.3. Behavior Change, Motivation, and Notifications

Self-monitoring alone may not be sufficient to maintain long-term engagement. Digital behavior-change interventions commonly combine goal setting, progress feedback, reminders, motivational messages, rewards, and personalized feedback to encourage continued participation [3].

Research indicates that the effectiveness of these techniques depends on how meaningfully they are implemented rather than simply on their presence. Personalization, understandable feedback, user-friendly interfaces, and appropriately timed reminders have also been associated with improved engagement and adherence [7], [10].

These findings are relevant to VitaMate's motivation and notification systems. Motivational feedback is therefore linked to verified user actions and progress rather than arbitrary rewards, while notifications are intended to reflect user preferences, schedules, and relevant tracking events.

3.4. Personalized and Constraints-Aware Health Support

Personalization adapts system behavior according to user characteristics, preferences, goals, and recorded information. In health applications, this may influence daily targets, reminders, progress indicators, and supportive recommendations.

In a multi-domain platform, personalization also creates a coordination problem. Chronic conditions, medications, personal goals, and tracker rules may impose overlapping or conflicting requirements. Managing these rules independently across modules may therefore produce inconsistent system behavior.

VitaMate addresses this issue through a centralized Constraints Layer that coordinates applicable health restrictions, goals, profile information, and tracker rules. Its role is to improve internal consistency rather than provide independent medical judgment.

3.5. AI-Assisted Food Analysis

Image-based food analysis aims to reduce the effort required for manual dietary logging by extracting food-related information from meal images. Typical approaches combine food-region detection or segmentation, food recognition, portion or weight estimation, and mapping the result to nutritional information [4].

This task remains challenging because meals may contain multiple or overlapping items, uncertain scale, mixed dishes, variable lighting, and partially visible ingredients. Recognition accuracy also does not guarantee accurate portion or nutrient estimation.

VitaMate therefore applies a user-in-the-loop approach. AI-generated food information is treated as provisional, allowing the user to review or correct uncertain results before final nutrition records are created.

Health-related digital systems must also account for data quality, privacy, access control, and responsible communication. Estimated or uncertain outputs should be distinguished from confirmed user data, while supportive guidance should not be presented as diagnosis or treatment [5].

3.6. Relevant Technologies and Current Trends

Modern mobile-health systems increasingly combine cross-platform mobile development, REST-based backend services, relational databases, computer-vision frameworks, and automated software-quality tools. Technologies relevant to the VitaMate domain include mobile frameworks such as Flutter, backend frameworks such as Django REST and FastAPI, relational data management, and deep-learning/computer-vision libraries used for intelligent food analysis. Exact technologies and versions used in VitaMate are documented in Chapter 6.

Table 9: Current Digital-Health Trends Relevant to VitaMate

Trend	Relevance to VitaMate
Personalization	Adapting goals, reminders and feedback
Connected Multi-Domain Data	Combining information across health trackers
AI-Assisted Interaction	Reducing manual food-logging effort
User-in-the-Loop Validation	Confirming uncertain AI outputs
Engagement-Oriented Design	Motivation, progress feedback and reminders
Privacy & Responsible Communication	Protecting health data and limiting unsupported claims

3.7. Review of Related Studies

Recent research has examined digital self-monitoring, personalization, medication adherence, user engagement, chronic-condition management, and image-based dietary assessment. The table below summarizes the studies most relevant to the design of VitaMate.

Table 10: Summary of Related Studies

Study	Focus	Key Finding	Relevance to VitaMate
Berry et al. [2]	Digital self-monitoring	Tailored digital tracking supported lifestyle-related improvements	Integrated tracking
Tong et al. [6]	Personalization	Personalized interventions showed positive lifestyle effects	Personalized goals
Aguiar et al. [3]	Behavior change	Goals, feedback and prompts can support adherence	Motivation

Study	Focus	Key Finding	Relevance to VitaMate
Jakob et al. [7]	mHealth adherence	Usability, personalization and reminders influence adherence	UI/UX + notifications
Belete et al. [8]	Medication reminders	Mobile reminders improved adherence in studied users	Notifications
Arshed et al. [9]	Medication adherence	Reminder effectiveness varied across interventions	Coordinated support
Kaur et al. [4]	Food-image AI	Recognition and nutrition estimation remain challenging	AI service
Milne-Ives et al. [10]	Engagement	Goals, feedback, prompts and rewards may support engagement	Motivation

Overall, the reviewed studies support the use of self-monitoring, personalization, reminders, and motivational feedback, while emphasizing that usability, implementation quality, and appropriate evaluation strongly influence outcomes. Food-image analysis also remains subject to recognition and estimation uncertainty. These findings support VitaMate’s integrated and user-controlled design without establishing clinical effectiveness for the system itself.

3.8. Comparative Analysis of Existing Applications

Several existing applications provide functions related to the VitaMate scope. MyFitnessPal focuses primarily on nutrition and fitness logging, Samsung Health provides a connected wellness view, Medisafe specializes in medication management, and Google Health provides activity, sleep, health-data tracking, and personalized coaching.

The comparison in the table below is based on the functions described by the official product sources. A limitation identified in the table represents a difference from the intended VitaMate scope rather than a general defect in the compared application.

Table 11: Comparison of Existing Health Applications

Application	Primary Focus	Main Strength	Gap Relative to VitaMate
MyFitnessPal	Nutrition & fitness	Detailed food and fitness tracking	Limited medication/chronic coordination
Samsung Health	General wellness	Broad health-data monitoring	No equivalent centralized constraints mechanism
Medisafe	Medication	Strong scheduling and adherence support	Limited multi-domain lifestyle tracking
Google Health	Activity, sleep & health data	Connected health information	Limited medication/nutrition-analysis coordination

The compared applications provide strong capabilities within their principal domains. Based on the functions reviewed in this report, however, none combines the complete intended VitaMate scope of multi-domain lifestyle tracking, medication support, centralized health constraints, coordinated motivation and notifications, and review-assisted food analysis.

3.9. Identified Research and System Gap

The reviewed literature commonly evaluates individual health behaviors or interventions, such as self-monitoring, medication adherence, engagement, or food-image analysis. Although these studies provide evidence for individual functions, they offer less guidance on coordinating several health domains within a unified application.

Existing applications similarly tend to provide either broad wellness tracking or specialized support for areas such as nutrition or medication management. Coordination among chronic-condition restrictions, personal goals, tracker behavior, notifications, motivational feedback, and intelligent services remains limited in the systems compared in this report.

VitaMate addresses this gap through an integrated architecture that combines multiple health and lifestyle domains with a centralized Constraints Layer, coordinated motivation and notifications, dashboards and reports, and review-assisted AI food analysis. Its contribution is therefore primarily architectural and integrative rather than diagnostic or clinically validated.

A second AI service for personalized dietary recommendations remained unimplemented in the submitted release. It is retained as future work and is not included in the implementation, testing, or results claims of the final system.

3.10. Chapter Summary

This chapter reviewed the theoretical and research foundations relevant to integrated health tracking, motivation, personalized coordination, notifications, and AI-assisted food analysis. Related studies and existing applications demonstrated the value of these individual capabilities while revealing limited coordination across multiple health domains. This gap provides the theoretical basis for VitaMate's integrated and constraints-aware design, which is analyzed through the system requirements in the following chapter.

Chapter 4:

Requirements

Analysis and System

Modeling

4.1 Introduction

This chapter defines the final VitaMate requirements baseline. It summarizes feasibility, requirement sources and stakeholders, consolidated functional and non-functional requirements, priority, UML and Agile requirements models, traceability, and representative initial test cases. Detailed specifications and supporting artifacts are moved to the appendices to keep the main report focused and readable.

4.2 Feasibility Study

VitaMate was evaluated from technical, economic, and operational perspectives to confirm that the approved scope could be implemented using the available technologies, team capabilities, and academic schedule.

Table 12: VitaMate Feasibility Assessment

Aspect	Assessment
Technical	Flutter, Django REST, PostgreSQL, Android services, and the AI runtime provide the required foundation. The main technical risks are cross-module integration, notification behavior, and AI uncertainty.
Economic	The academic implementation relies mainly on open-source technologies and existing development hardware; no unsupported commercial cost or revenue claims are made.
Operational	The application uses familiar mobile interactions, editable records, reminders, progress views, and user confirmation. It is a self-monitoring and lifestyle-support system rather than a medical diagnostic tool.

4.3 Requirements Elicitation and Sources

The baseline created during Senior Project 1 was refined during Senior Project 2 using implementation findings, supervisor feedback, scientific literature, related-application analysis, backlog refinement, and testing results. No user surveys or interviews are claimed as requirement sources.

Table 13: Requirements Sources

Source	Contribution
Senior Project 1 documentation	Initial scope, requirements, use cases, constraints, and traceability baseline
Scientific literature	Safety, personalization, behavior-change, usability, notification, and AI considerations

Source	Contribution
Related applications	Feature comparison and system-gap identification
Supervisor feedback	Scope control, technical refinement, and academic quality
Implementation and testing	Defects, integration needs, reliability, and usability requirements
Product Backlog	Prioritization and final-scope refinement

4.4 Stakeholders and Needs Analysis

The final project has no external commercial client or separate human administrator. The principal stakeholders are the end user, academic supervisors, development team, and future maintainers.

Table 14: Stakeholders and Main Needs

Stakeholder	Main Need
Regular User	Secure, simple, consistent tracking with understandable feedback and privacy protection
Academic Supervisors	Correct scope, evidence-based decisions, technical quality, and clear documentation
Development Team	Clear requirements, stable integration contracts, and maintainable modules
Future Maintainers	Readable code, documented APIs, controlled versions, and reproducible environments

Only requirements that were relevant to the approved scope, technically feasible, and traceable to a stakeholder need or system dependency were included in the final baseline.

4.5 Functional Requirements

Closely related operations are consolidated into domain-level requirements to avoid unnecessary fragmentation while preserving the complete approved system scope.

Table 15: Consolidated Functional Requirements of the VitaMate System

ID	Area	Functional Requirement
FR-01	Account & Profile	Register, authenticate, sign out, and maintain a secure authenticated session.
FR-02	Account & Profile	View and update personal and health-profile data, preferences, objectives, profile image, and delete the account through confirmation.
FR-03	Dashboard	Provide a daily home dashboard summarizing enabled health and lifestyle modules with direct access to their main functions.

ID	Area	Functional Requirement
FR-04	Targets	Calculate applicable daily targets from profile and health information and recalculate affected targets when relevant data changes.
FR-05	Health Score	Calculate and display a health score from defined health and tracker indicators independently from motivational points.
FR-06	Cross-Domain Coordination	Coordinate goals, health indicators, chronic-condition restrictions, and tracker rules through the Constraints Layer and update affected outputs.
FR-07	Activity	Obtain supported step data with permission and display steps, goal progress, estimated distance, active time, and calories.
FR-08	Activity	Configure, start, pause, resume, and stop live activity sessions and display relevant session information.
FR-09	Activity	Manage activity records, goals, favorites, suggestions, and history with recalculation after changes.
FR-10	Nutrition	Calculate nutrition targets and display calorie, macronutrient, and available micronutrient progress.
FR-11	Nutrition	Record meals manually, search the food catalog, access favorite/recent foods, and review meal details and history.
FR-12	Nutrition	Update/delete meals, recalculate totals, and display applicable dietary warnings.
FR-13	AI Food Analysis	Capture or upload a supported meal image and validate it before AI processing.
FR-14	AI Food Analysis	Identify supported food items, estimate quantities when sufficient information is available, and map candidates to the nutrition source.
FR-15	AI Food Analysis	Present provisional AI-generated food and nutrition information with relevant confidence or limitation information.
FR-16	AI Food Analysis	Allow review, editing, confirmation, or rejection of AI results; only confirmed results are finalized.
FR-17	Hydration	Calculate and adjust hydration targets from profile information, activity, and applicable constraints.
FR-18	Hydration	Record/manage water and supported beverages, calculate contribution, display progress, and configure reminders.
FR-19	Sleep	Create, update, and delete sleep records and calculate sleep duration/history.

ID	Area	Functional Requirement
FR-20	Sleep	Define sleep goals and schedules, configure sleep/wake reminders, and review related progress.
FR-21	Medication	Create, update, and remove medications and associate them with a registered chronic condition when applicable.
FR-22	Medication	Define dose schedules, frequency, duration, instructions, refill information, and applicable reminders.
FR-23	Medication	Record scheduled doses as taken, skipped, or missed and maintain adherence history without modifying treatment.
FR-24	Chronic Conditions	Manage supported conditions, record applicable readings, and review history/trends.
FR-25	Conditions & Constraints	Generate and apply condition-related restrictions, warnings, target adjustments, and coordinated condition summaries.
FR-26	Goals	Create/manage measurable goals, targets, duration, completion criteria, reminders, and linked progress.
FR-27	Habits	Create gradual-reduction or quitting plans and record adherence or relapse while preserving progress history.
FR-28	Habits	Display progress, streaks and motivational feedback, connect relevant trackers, and manage plan state.
FR-29	Notifications	Provide centralized notification settings for categories, schedules, frequency, quiet hours, and required permissions.
FR-30	Notifications	Coordinate notification plans with local-device scheduling and synchronize/cancel/reschedule/restore them after relevant changes.
FR-31	Motivation	Award motivational points for defined verified actions while preventing duplicate rewards for the same confirmed action.
FR-32	Motivation	Maintain motivational transaction history and update daily completion, missions, streaks, achievements, levels, and feed events.
FR-33	Reports	Generate daily, weekly, and periodic summaries from verified health and lifestyle records.
FR-34	Integrated Progress	Provide coordinated dashboards, trends, indicators, warnings, and supportive guidance without diagnostic or treatment claims.

The approved SRS contains 96 atomic functional requirements. For readability, the main report consolidates the implemented behavior into the requirements above, while Appendix C preserves

SRS-level traceability and final implementation status. The only SRS functional group not implemented in the submitted release is the second AI personalized dietary recommendation service (FR-60-FR-66).

Table 16: Approved SRS Functional-Domain Implementation Status

SRS IDs	Functional Domain	Final Status
FR-01-FR-05	Profile Management	Completed
FR-06-FR-16	Physical Activity Tracking	Completed
FR-17-FR-24	Nutrition Tracking	Completed
FR-25-FR-29	Hydration Management	Completed
FR-30-FR-33	Sleep Tracking	Completed
FR-34-FR-36	Motivation and Reporting	Completed
FR-37-FR-39	Personal Goals Management	Completed
FR-40-FR-46	Medication Management	Completed
FR-47-FR-53	Unhealthy Habit Management	Completed
FR-54-FR-59	AI-Based Food Analysis	Completed with mandatory user review
FR-60-FR-66	AI Personalized Diet Recommendation	Deferred - not implemented in submitted release
FR-67-FR-80	Chronic Condition Support	Completed
FR-81-FR-86	Beverage Tracking	Completed
FR-87-FR-96	Vitamins and Minerals Monitoring	Completed

4.6 Non-Functional Requirements

The non-functional requirements define the main quality attributes expected from the final implementation and are linked to verifiable acceptance criteria used in Chapter 7.

Table 17: Non-Functional Requirements of the VitaMate System

ID	Category	Requirement	Acceptance Criterion
NFR-01	Performance	Core non-AI services complete the defined automated and integration workflows without unhandled timeouts.	All selected core workflow requests complete successfully in the documented test environment; targeted dashboard/history latency is reported in Chapter 7.
NFR-02	Security	Protected functions require valid authentication and authorization.	Unauthorized and cross-user access attempts are rejected.
NFR-03	Privacy	Users access only data associated with their own account.	No tested user retrieves another user's protected records.

ID	Category	Requirement	Acceptance Criterion
NFR-04	Usability	Core workflows provide clear navigation, validation, loading, and error feedback.	Defined tasks complete without ambiguous system states.
NFR-05	Reliability	Repeated/failed requests do not create duplicate or incomplete confirmed records.	Retry scenarios preserve one valid final record.
NFR-06	Data Integrity	Dashboard, tracker, and report values remain consistent with source records.	Regression outputs match verified underlying data.
NFR-07	Maintainability	Frontend, backend, data, and intelligent services remain modular.	Module boundaries are preserved without unresolved critical static-analysis errors.
NFR-08	Compatibility	Main mobile workflows operate in the defined Android environment.	Selected acceptance workflows complete in the supported environment.
NFR-09	Notification Reliability	Scheduling remains consistent with preferences and relevant data changes.	Schedule/cancel/reschedule cases behave correctly.
NFR-10	AI Transparency	AI meal results remain provisional until user review.	Every result supports correction, rejection, or confirmation before finalization.
NFR-11	Health Safety	Supportive outputs are not presented as diagnosis or treatment.	Interfaces avoid diagnostic or medication-changing instructions.
NFR-12	Recoverability	Connection/service failures do not corrupt confirmed data.	Confirmed records remain unchanged after simulated failures.

4.7 Requirements Prioritization Using MoSCoW

MoSCoW prioritization controls the final scope at feature-group level. Priority indicates importance to the release rather than implementation status.

Table 18: MoSCoW Prioritization of VitaMate Requirements

Priority	Scope
Must Have	Authentication/profile, core trackers, medication, chronic-condition constraints, cross-domain coordination, notification scheduling, AI review/confirmation, core dashboards/reports, and data protection.
Should Have	Motivation features, streaks/achievements, favorites/quick access, UI/UX refinements, and enhanced progress views.
Could Have	Additional motivational-feed refinements and optional personalized supportive guidance.
Won't in Current Release	Medical diagnosis, treatment prescription or dose modification, clinical decision support, unrestricted wearable integration, and automatic saving of unconfirmed AI results.

4.8 UML Requirements Models

The main report includes only the UML models needed to explain the principal user goals and the most important Senior Project 2 workflows. Detailed use-case specifications and additional UML models are provided in Appendices A and B.

4.8.1 Main Use Cases

Table 19: Main Use Cases of VitaMate

ID	Use Case	Main Scope
UC-01	Manage Account and Health Profile	Authentication, profile, health data, and preferences
UC-02	Manage Daily Health Tracking	Activity, steps, hydration, and sleep tracking
UC-03	Manage Nutrition and AI-Assisted Meals	Manual meals, nutrition tracking, AI analysis, and confirmation
UC-04	Manage Medications and Chronic Conditions	Schedules, adherence, conditions, readings, and restrictions
UC-05	Manage Goals and Unhealthy Habits	Goals, behavior-change plans, and progress
UC-06	Manage Notifications and Motivational Support	Preferences, reminders, points, missions, streaks, and feedback
UC-07	View Reports and Supportive Guidance	Dashboard, reports, trends, and coordinated guidance

4.8.2 Overall Use Case Diagram

The high-level use case model focuses on the seven principal user goals; internal technical services are intentionally omitted for readability.

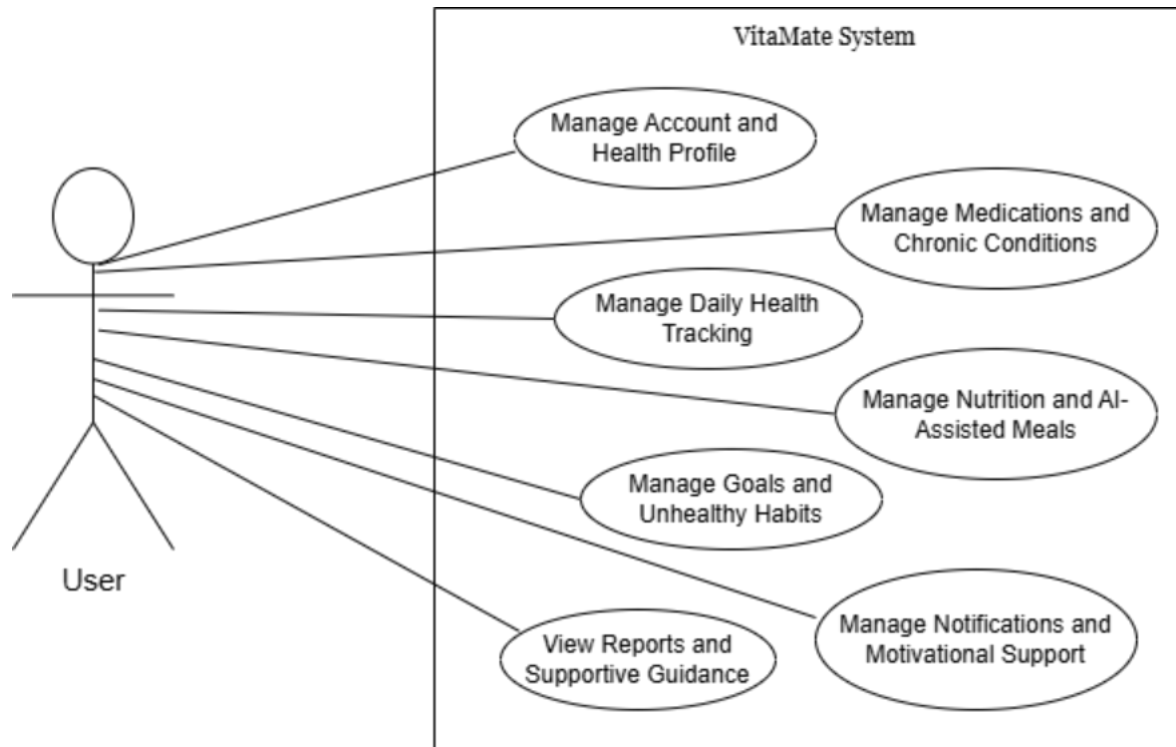


Figure 2: High-Level Use Case Diagram of VitaMate

4.8.3 Summary of Main Use Case Specifications

Table 20: Summary of Main Use Case Specifications

UC	Actor	Precondition	Main Interaction	Main Alternative
UC-01	User	System available	Authenticate and manage profile	Invalid credentials/data
UC-02	User	Authenticated; permissions when required	Record/review daily tracker data	Invalid data or device failure
UC-03	User	Authenticated; valid meal/image	Record or analyze, review, and confirm meal	Invalid/low-confidence result
UC-04	User	Authenticated	Manage medications/conditions and related restrictions	Invalid/conflicting data
UC-05	User	Authenticated	Manage goals/habits and progress	Pause, relapse, or invalid target
UC-06	User	Authenticated; notification permission when required	Configure reminders and receive motivation	Permission, quiet-hour, or duplicate-event case
UC-07	User	Authenticated	Review dashboards, reports, and guidance	Missing data or loading failure

Detailed triggers, inputs, outputs, preconditions, postconditions, main flows, and alternative flows are provided in Appendix A: Detailed Use Case Specifications.

4.8.4 Selected Activity Diagrams

The selected activity diagrams emphasize the AI-assisted meal workflow and the motivation update triggered by a verified tracker action. Additional behavior-change modeling is retained in Appendix B.



Figure 3: Activity Diagram for AI-Assisted Meal Analysis

Activity Diagram for Health-Tracker Recording and Motivation-State Update

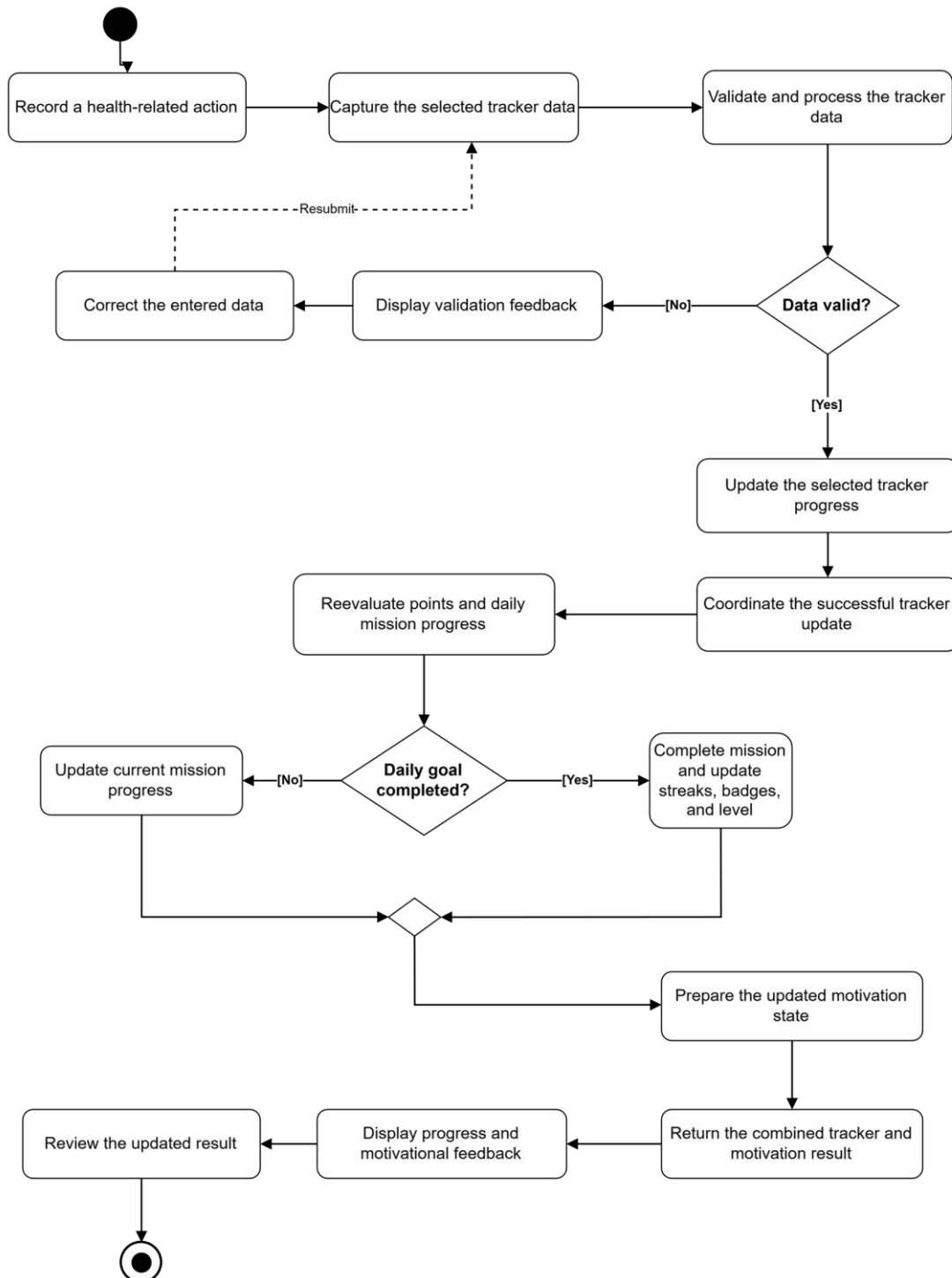


Figure 4: Activity Diagram for Health-Tracker Recording and Motivation-State Update

4.8.5 Selected Sequence Diagrams

The sequence diagrams show the principal service interactions for AI-assisted meal analysis and motivation updates. Technical notification design and implementation are presented in Chapters 5 and 6.

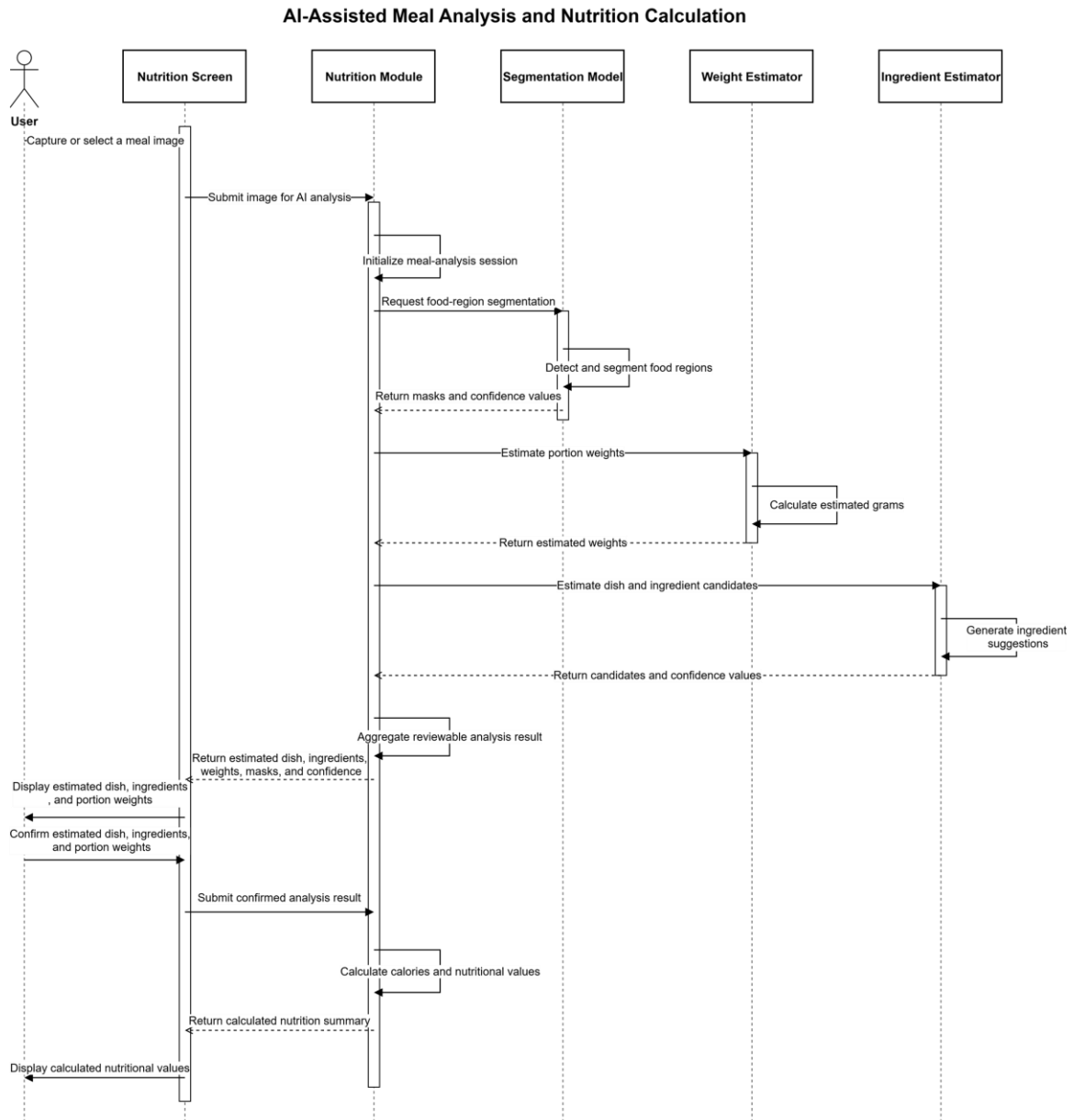


Figure 5: Sequence Diagram for AI-Assisted Meal Analysis

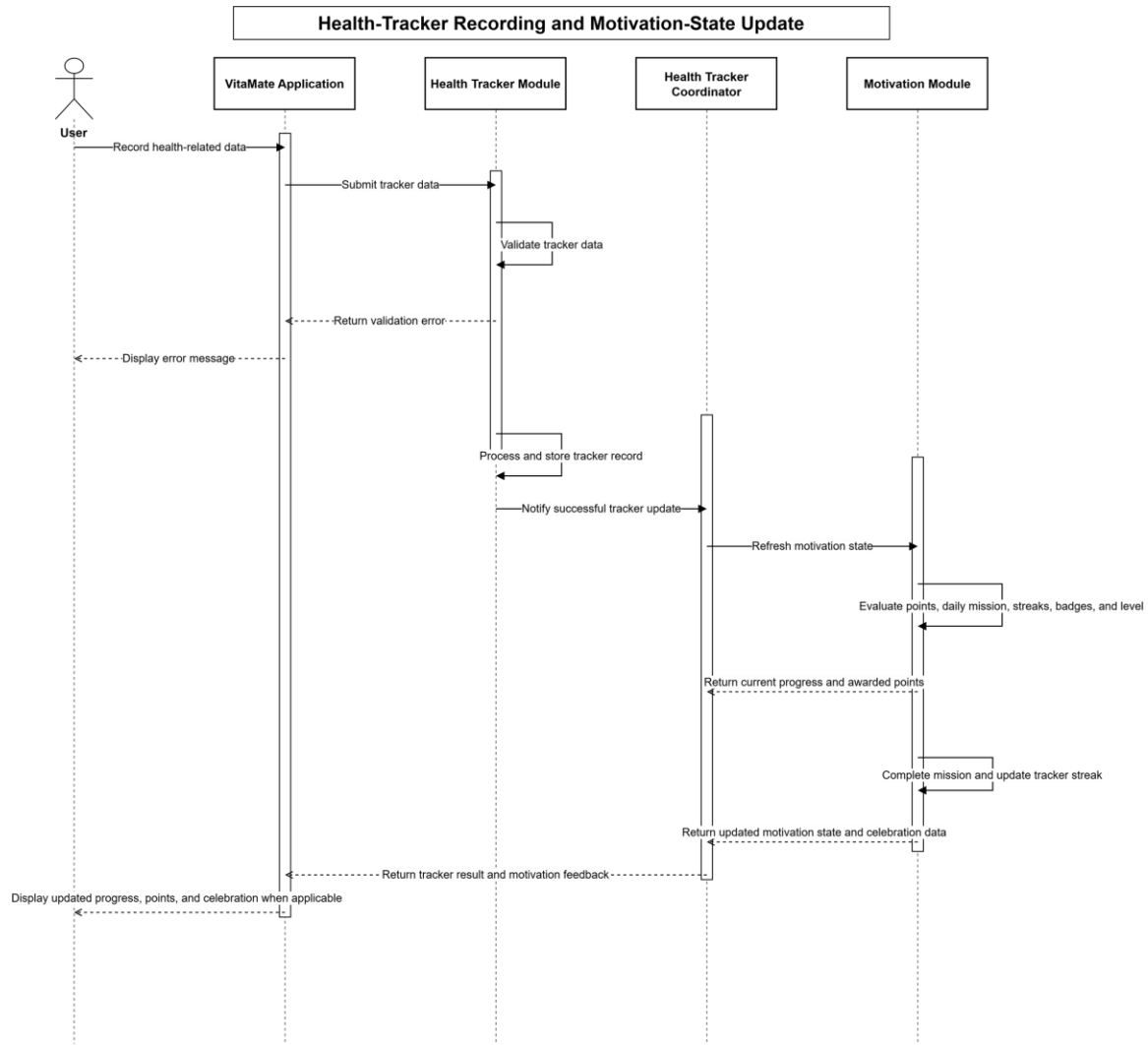


Figure 6: Sequence Diagram for Health-Tracker and Motivation Update

4.9 Agile Requirements Models

Representative Agile artifacts summarize how user needs were translated into prioritized Senior Project 2 work. Detailed supporting artifacts are retained in Appendix B where they add information beyond the summaries below.

4.9.1 User Stories

Table 21: Representative VitaMate User Stories

ID	User Story
US-01	As a user, I want to manage my health profile so that VitaMate can personalize my tracking experience.

ID	User Story
US-02	As a user, I want to record and review daily health data so that I can monitor my progress.
US-03	As a user, I want to analyze a meal image so that food logging requires less manual effort.
US-04	As a user, I want to review and correct AI-generated meal results before they are saved.
US-05	As a user, I want my health conditions and goals to be considered across related trackers.
US-06	As a user, I want to manage medication schedules and adherence records.
US-07	As a user, I want to control notification preferences and quiet hours so that reminders are delivered appropriately.
US-08	As a user, I want meaningful motivational feedback based on my verified progress.
US-09	As a user, I want clear and consistent interfaces so that daily tracking is simple and understandable.
US-10	As a user, I want to review dashboards and reports so that I can understand my overall progress.

4.9.2 Story Mapping

Table 22: VitaMate User Story Map

User Journey	Main User Goal	Related Stories
Personalize	Configure personal and health information	US-01, US-05
Track Daily Health	Record and monitor health-related activities	US-02, US-06
Analyze Nutrition	Record meals manually or through AI assistance	US-03, US-04
Receive Support	Receive appropriate reminders and motivation	US-07, US-08
Use the Application Easily	Complete daily workflows through clear interfaces	US-09
Review Progress	Review dashboards, reports, and trends	US-10

4.9.3 Product Backlog

All principal Product Backlog items summarized below were completed within the final Senior Project 2 scope.

Table 23: Senior Project 2 Product Backlog Summary

ID	Product Backlog Item	Priority	Final Status
PB-01	Core system and tracker integration	High	Completed

ID	Product Backlog Item	Priority	Final Status
PB-02	Constraints and cross-domain coordination	High	Completed
PB-03	Motivation-system development and integration	High	Completed
PB-04	Notification-system development and integration	High	Completed
PB-05	UI/UX redesign and usability refinement	High	Completed
PB-06	AI food-analysis development and nutrition integration	High	Completed
PB-07	System testing, defect correction, and stabilization	High	Completed
PB-08	Documentation and final academic delivery	High	Completed
PB-09	Second AI personalized dietary recommendation service (SRS FR-60-FR-66)	High	Deferred - Future Work

4.9.4 Sprint Backlog

The Sprint Backlog evolved with integration findings and technical dependencies. The final summary emphasizes sprint objectives and delivered work; detailed calendar dates are documented in the project Gantt chart rather than duplicated here.

Table 24: VitaMate Sprint Backlog Summary

Sprint	Main Objective	Principal Deliverables
Sprint 1	Planning and integration baseline	Scope refinement, dependency review, integration preparation
Sprint 2	Core development	Motivation and notification development; selected cross-module integration
Sprint 3	AI and UI/UX refinement	AI food-analysis development/integration; interface redesign and workflow refinement
Sprint 4	System integration and stabilization	Cross-module coordination; notification/motivation integration; defect correction
Sprint 5	Final validation and delivery	Testing, stabilization, documentation, final demonstration, and delivery

4.10 Requirements Traceability and Initial Test Cases

Traceability links each requirement area with the design/implementation component that realizes it and the verification activity used to evaluate it. A summarized matrix is kept in the main

report; the detailed requirement-level matrix is provided in Appendix C together with detailed test cases and evidence.

4.10.1 Requirements Traceability Matrix

Table 25: VitaMate Requirements Traceability Summary

Requirement Area	Related Requirements	Main Component	Verification
Account & Profile	FR-01–FR-02	Authentication / Profile	API and security tests
Dashboard & Coordination	FR-03–FR-06	Dashboard / Coordinator / Constraints	Integration and regression tests
Activity	FR-07–FR-09	Activity / Device Services	Functional and device tests
Nutrition	FR-10–FR-12	Nutrition	Functional and integration tests
AI Food Analysis	FR-13–FR-16	AI Service / Nutrition Integration	AI, integration, and performance tests
Hydration & Sleep	FR-17–FR-20	Hydration / Sleep	Functional and regression tests
Medication & Conditions	FR-21–FR-25	Medication / Conditions / Constraints	Functional and integration tests
Goals & Habits	FR-26–FR-28	Goals / Habits / Tracker Coordination	Functional and motivation tests
Notifications	FR-29–FR-30	Notification Hub	Scheduling and integration tests
Motivation	FR-31–FR-32	Motivation Services	Unit and integration tests
Reports & Progress	FR-33–FR-34	Reporting / Health Coordinator	Integration and data-integrity tests
Personalized AI Recommendation	SRS FR-60–FR-66	Planned Recommendation Service	Not executed - service deferred to future work

4.10.2 Initial Test Cases

Representative initial cases establish the expected behavior before execution results are discussed in Chapter 7. The table intentionally excludes final pass/fail values to keep requirements analysis separate from test evaluation.

Table 26: Representative Initial Test Cases

ID	Area	Test Case	Expected Result
ITC-01	Authentication	Authenticate with valid credentials	User is authenticated and protected functions become available
ITC-02	Tracker Integration	Create a valid tracker record	Record is stored and related progress values are updated

ID	Area	Test Case	Expected Result
ITC-03	Constraints	Modify health data affecting a rule	Applicable target/warning is recalculated
ITC-04	Motivation	Process the same eligible confirmed action twice	Reward is applied once only
ITC-05	Notifications	Modify a saved notification schedule	Previous plan is replaced and synchronized correctly
ITC-06	AI Food Analysis	Submit and review an AI-generated meal result	Result remains provisional until confirmation
ITC-07	Data Integrity	Update/delete a source tracker record	Related summaries are recalculated consistently
ITC-08	Reports	Request an integrated progress summary	Verified records are aggregated into expected indicators
ITC-09	Personalized AI Recommendation	Generate a personalized dietary recommendation from user health/lifestyle data	Deferred in the submitted release; no execution result is claimed

4.11 Chapter Summary

This chapter established the final VitaMate requirements baseline through feasibility assessment, requirement sources, stakeholders, consolidated requirements, prioritization, UML and Agile models, traceability, and representative initial test cases. Supporting detail is retained in the appendices, while the next chapter presents the architectural and detailed design.

Chapter 5:

Architectural and Detailed Design

5.1 Introduction

This chapter presents the architecture and detailed design decisions that support the final VitaMate implementation. The design emphasizes modular backend services, coordinated health data, controlled notification delivery, user-centered mobile interaction, and review-assisted AI food analysis.

5.2 High-Level Architecture and Architectural Pattern

VitaMate follows a layered client–server architecture. The Flutter Android application provides the user interface, while the Django REST backend owns authentication, business rules, cross-module coordination, and persistent-data control. PostgreSQL stores canonical application data. A separate FastAPI runtime performs meal-image analysis and is accessed only through Django.

The architecture separates presentation, API handling, domain services, data access, and intelligent processing. This structure reduces duplication, supports testing, and allows tracker-specific modules to share common coordination services without coupling the mobile client to internal AI or database details.

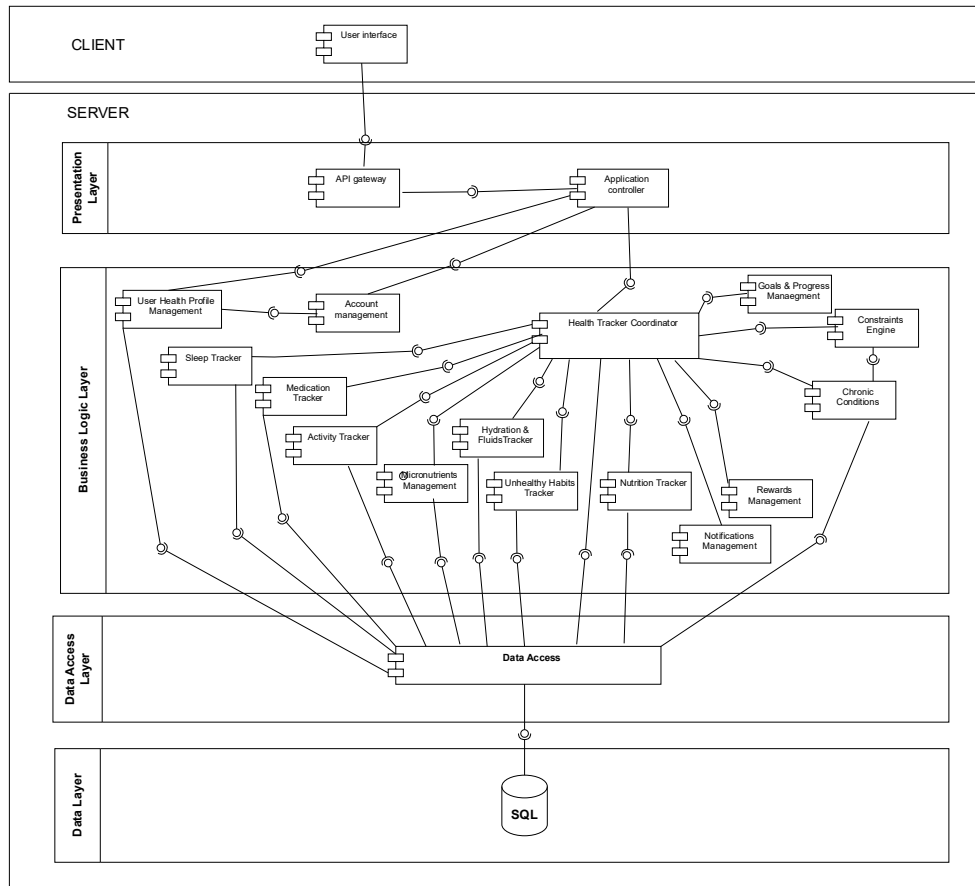


Figure 7: Final High-Level Architecture of VitaMate

5.3 Data Design

The relational data model groups persistent entities by functional domain while retaining user ownership and shared relationships required for coordinated health tracking. The ERD below presents the main entities and relationships; detailed field-level schemas and migration/configuration evidence are moved to Appendix E.

The ERD retains the documented diet-plan relation from the approved design baseline for traceability. That relation belongs to the deferred second AI recommendation service and is not treated as an implemented final-release component. The representative relational schema in Appendix E distinguishes implemented relations from this deferred design element.

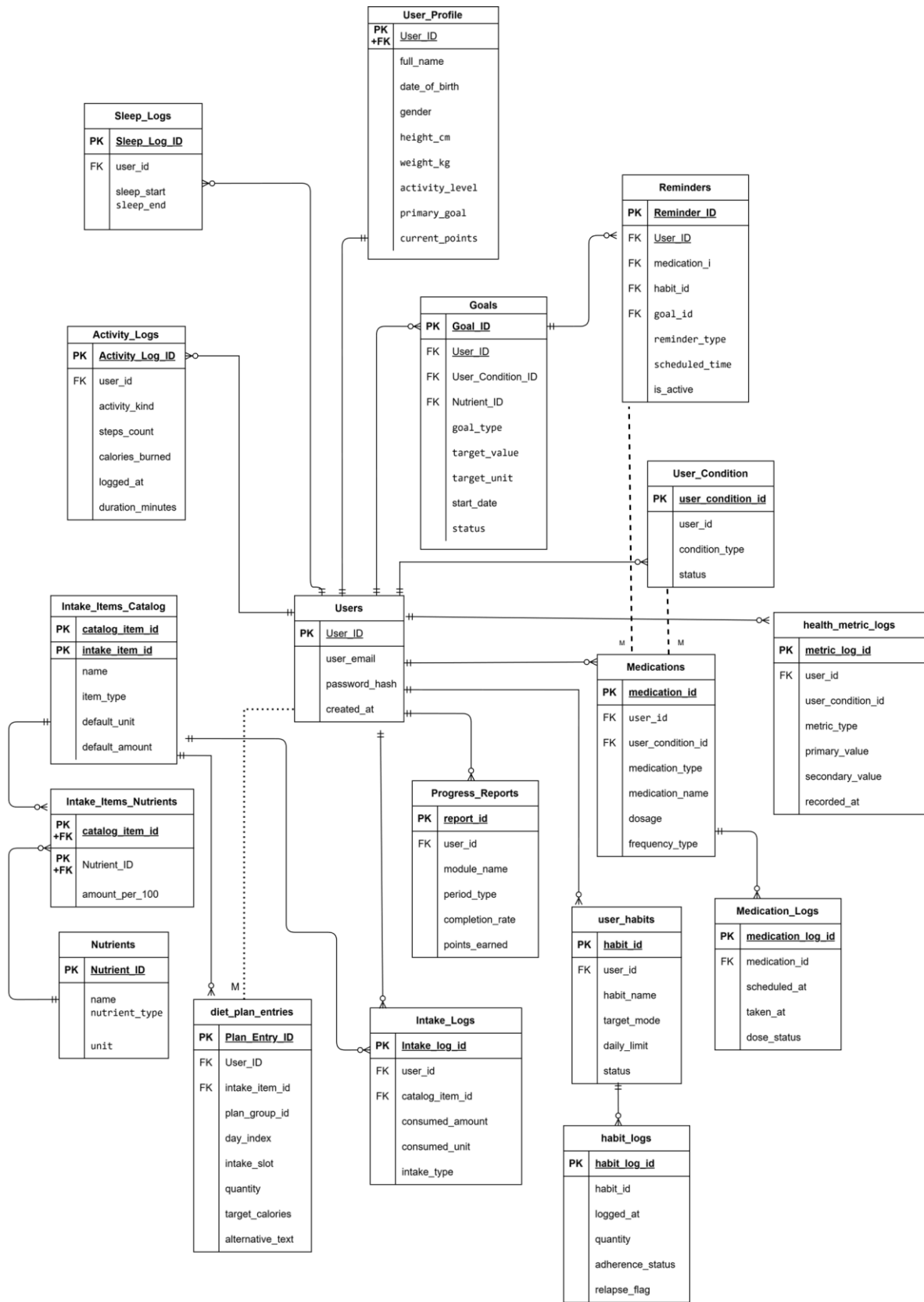


Figure 8: Entity–Relationship Diagram of VitaMate

Table 27: Main Database Domains and Responsibilities

Domain	Representative Data
Users & Profiles	Authentication identity, health profile, preferences
Nutrition	Food catalog, meals, meal components, nutrient snapshots
Hydration	Water/beverage records, targets, progress
Activity & Sleep	Activity sessions, steps, sleep records, goals
Medication	Medication plans, schedules, dose logs, adherence
Chronic Conditions	Conditions, health readings, related constraints
Goals & Habits	Goals, habit plans, logs, progress
Motivation	Points/transactions, missions, streaks, achievements, events
Notifications	Devices, preferences, plans, synchronization/delivery state
AI Meals	Provisional analysis sessions and review/finalization state

5.4 API and Security Design

5.4.1 API Structure and Conventions

The mobile application communicates with the Django backend through REST APIs. Protected requests use JWT authentication. API views validate and serialize requests, delegate business logic to services, and return endpoint-specific JSON responses and standard HTTP status codes. Detailed endpoint extracts and longer API documentation are provided in Appendix E.

Table 28: General API Design Conventions

Aspect	Design
Transport	REST over HTTP/HTTPS according to the active environment
Authentication	JWT access and refresh tokens for protected endpoints
Validation	Serializer/request validation before business processing
Ownership	User-owned records are scoped to the authenticated user
Errors	4xx for invalid/unauthorized requests; 5xx/503 for service failures where applicable
AI Integration	Flutter calls Django; Django calls the internal FastAPI runtime

Table 29: Main VitaMate API Groups

API Group	Principal Functions
Authentication & Profile	Registration, login, profile/health data, preferences
Health Trackers	Nutrition, hydration, activity, steps, sleep
Medication & Conditions	Medication plans/logs, conditions, readings, constraints
Goals & Habits	Goals, behavior-change plans, progress and history

API Group	Principal Functions
Motivation	Points, missions, streaks, achievements, motivational feed
Notifications	Device registration, preferences, plan synchronization
Progress & Reports	Home/progress summaries and coordinated read models
AI-Assisted Meals	Analyze, review, confirm/reject, and finalize meal sessions

5.4.2 Authentication, Authorization, and Data Protection

Django and Django REST Framework enforce authentication and ownership at the backend. JWT credentials are stored by the Flutter client using secure storage. Protected records are filtered by authenticated-user ownership, and sensitive credentials are excluded from version control. The design also separates health-support outputs from diagnostic or treatment decisions.

Security testing and user-isolation evidence are presented in Chapter 7; detailed configuration evidence is retained in Appendix E.

5.4.3 AI Food-Analysis Interface

The AI interface uses an analyze–review–finalize workflow. Django validates the authenticated request and image, forwards the image to the internal FastAPI runtime, stores the returned result as a temporary analysis session, and prevents final meal creation until the user confirms the reviewed components and quantities.

Table 30: Representative AI-Meal API Contract

Item	Design
Analyze request	Authenticated multipart meal-image submission to Django
AI processing	Internal Django-to-FastAPI request
Provisional response	Dish/component candidates, confidence/limitations, review state
User action	Edit, confirm, or reject provisional result
Finalization	Django maps confirmed components to the canonical nutrition catalog
Failure handling	Invalid input is rejected; service failures do not create a final meal

5.5 Senior Project 2 Component Design

Senior Project 2 concentrated on four design areas: motivation, notifications, UI/UX refinement, and AI food analysis. The following sections document their final responsibilities and integration boundaries.

5.5.1 Motivation System Design

The motivation system is separated from the health score and reacts only to verified user actions. A successfully processed tracker event can update motivational points, mission progress, streaks, achievements, levels, and feed events. Duplicate-reward protection prevents the same confirmed action from being rewarded more than once.

Table 31: Motivation-System Design Responsibilities

Design Element	Responsibility
Verified Action Input	Receives eligible events only after successful domain processing
Point/Progress State	Maintains motivational transactions and accumulated progress
Missions & Streaks	Re-evaluates progress after relevant verified actions
Achievements & Levels	Updates milestone state when defined conditions are reached
Motivation Feed	Presents user-facing feedback without changing the independent health score

5.5.2 Notification System Design

Notifications follow a backend-planned and locally delivered design. The backend stores registered devices and user preferences, evaluates applicable reminder rules and quiet hours, and generates notification plans. Flutter synchronizes these plans and uses the Android local-notification service for device delivery. Relevant synchronization and delivery state can be reported back to the backend.

Table 32: Notification-System Design Responsibilities

Component	Responsibility
Device Registry	Associates the supported mobile device with the authenticated user
Preferences	Stores enabled categories, timing, frequency, and quiet-hour choices
Plan Generation	Produces the applicable notification plan from current rules and preferences
Flutter Synchronization	Retrieves plan changes and aligns local scheduled notifications
Android Scheduler	Schedules/cancels local reminders on the device
Delivery State	Supports resynchronization after relevant app, device, or preference changes

5.5.3 UI/UX Design

The Flutter interface was redesigned to reduce clutter, improve navigation, and create a consistent visual identity across health modules. Feature screens use shared components and named routes, while data-driven interfaces expose clear loading, empty, error, retry, and successful-content states.

Table 33: UI/UX Design Principles Applied in Senior Project 2

Principle	Application
Clarity	Short labels, focused cards, and prioritized information
Consistency	Shared visual components, typography, spacing, and interaction patterns
Low Friction	Quick access to frequent logging and review actions
State Visibility	Loading, validation, empty, error, retry, and success feedback
User Control	Editable health records, configurable notifications, and reviewable AI results

5.5.4 AI Food-Analysis Design

The final intelligent service is limited to AI-assisted meal-image analysis. Flutter does not call the AI runtime directly; Django remains the authenticated gateway and authority for provisional-session ownership and final meal creation.

The FastAPI runtime produces confidence-scored dish and component suggestions. Django stores them in an expiring provisional session. The user can correct the dish, included components, mappings, and gram values before confirmation. Raw AI output is never the authoritative source of final nutritional values.

After confirmation, Django maps the reviewed components to VitaMate’s canonical nutrition catalog and calculates final calories, macronutrients, vitamins, and minerals. This design separates visual estimation from trusted nutrition calculation and provides a safe fallback for uncertain predictions.

Table 34: AI Food-Analysis Design Responsibilities

Component	Design Responsibility
Flutter application	Uploads the image and provides review/correction interfaces
Django REST backend	Authenticates, validates, stores provisional sessions, and controls finalization
FastAPI runtime	Processes the image and returns confidence-scored suggestions

Component	Design Responsibility
Analysis session	Stores provisional result, review state, and expiry information
Nutrition catalog	Provides canonical values used for final nutrition calculation
User confirmation	Prevents unreviewed predictions from becoming final meal records

5.6 Cross-Module Coordination

The Health Tracker Coordinator provides unified read-side information for home, progress, and history views, while the rule-based Constraints Layer resolves applicable goals, restrictions, and tracker targets. Shared coordination services reduce duplicated logic and keep motivational points distinct from the health score. The design is integrative rather than diagnostic.

5.7 Chapter Summary

This chapter presented the final VitaMate design: a Flutter client, layered Django REST backend, PostgreSQL database, backend-planned notification workflow, coordinated health services, and a separate review-assisted FastAPI food-analysis runtime. The design gives particular emphasis to the four Senior Project 2 areas and provides the structure implemented in Chapter 6.

Chapter 6:

Development

Environment and

Software

Implementation

6.1 Introduction

This chapter documents the verified development environment and implementation of VitaMate, with primary emphasis on the four major Senior Project 2 developments: the motivation system, notification system, UI/UX refinement, and AI food-analysis service. Earlier implementation is summarized only as the baseline required to understand these changes.

6.2 Senior Project 1 Implementation Baseline

By the end of Senior Project 1, VitaMate already included the Flutter mobile foundation, Django REST backend, PostgreSQL data layer, authentication/onboarding, home interface, and core nutrition, hydration, activity/steps, sleep, chronic-condition, and medication modules. Senior Project 2 used this baseline for integration, redesign, extension, and final validation.

6.3 Development Environment and Engineering Setup

The project used a Windows-based local development environment together with Git/GitHub and GitHub Actions for repeatable verification. Exact versions are reported where they were pinned or directly verified; unpinned AI/runtime tools are identified without inventing version numbers.

Table 35: Main Development Tools and Verified Versions

Tool / Library	Version / Constraint	Purpose
Dart	^3.10.1	Flutter application language
Flutter	>= 3.38.0; CI stable	Mobile framework
Python	3.10.0 local; 3.13 CI	Backend and AI runtime
Django	5.2.10	Backend framework
Django REST Framework	3.16.1	REST APIs
SimpleJWT	5.5.1	JWT authentication
PostgreSQL	17.9-3 local; 17 CI	Relational database
Psycopg	>=3.2,<4	PostgreSQL driver
Pillow	>=10.4,<13	Backend image processing
Dio	5.9.0	Flutter HTTP communication
Flutter Secure Storage	9.2.4	JWT storage
Flutter Local Notifications	17.2.4	Android local scheduling
Image Picker	1.2.3	Meal-image capture/selection
FastAPI / Uvicorn	Not pinned	Internal AI runtime

Tool / Library	Version / Constraint	Purpose
Transformers	5.5.4	AI model loading/inference
PyTorch / YOLO / OpenCV / NumPy	Not fully pinned	Meal-image analysis
GitHub Actions	Repository workflow	Continuous integration

Table 36: Engineering Environment Summary

Area	Implementation
Source Control	Git repository hosted on GitHub; main branch used for final integration. A formal feature-branch naming policy was not documented.
Continuous Integration	Automated secret scanning, backend tests, Flutter analysis/tests, Android-emulator integration where enabled, and artifact upload. No automated production deployment was implemented.
Backend Test Environment	Django/DRF test runner with PostgreSQL-compatible settings and reusable seed/reference data.
Frontend Test Environment	Flutter unit, widget, golden, compact-screen, and integration_test tooling.
AI Test Environment	Runtime smoke tests and pipeline evaluation for the release image set.

Detailed folder trees, dependency/configuration files, CI configuration, and representative code extracts are provided in Appendix E.

6.4 Senior Project 2 Implementation

6.4.1 Motivation System Implementation

The motivation subsystem was developed as a dedicated backend domain rather than a visual-only points counter. Verified tracker actions are processed by their domain services first; only successful operations can trigger motivational updates. This prevents motivational state from becoming inconsistent with the source health records.

The subsystem maintains point/progress history and updates missions, streaks, achievements, levels, and motivation-feed events. Duplicate-reward protection is applied so that repeated processing of the same confirmed action does not create a second reward. Motivation remains separate from the daily health score, which is calculated from health/tracker indicators rather than reward points.

Table 37: Motivation Implementation Flow

Stage	Implementation Role
1. Domain Validation	Tracker/service validates and persists the user action

Stage	Implementation Role
2. Eligibility Check	Only eligible verified actions continue to motivation processing
3. Duplicate Protection	Repeated reward for the same confirmed action is prevented
4. Progress Update	Points and applicable mission/streak state are updated
5. Milestone Update	Achievements/levels are re-evaluated where applicable
6. User Feedback	Motivational event/feed state is returned for presentation in Flutter

Integration with trackers makes the motivation system event-driven from actual user progress rather than isolated manual scoring. The same design also supports habit progress and milestone feedback while preserving historical state.

6.4.2 Notification System Implementation

The notification feature was redesigned from a primarily frontend reminder mechanism into a coordinated backend-to-device workflow. Backend services maintain device ownership, notification preferences, and generated plans. Preferences include enabled categories, schedules, frequencies, and quiet-hour behavior where applicable.

Flutter synchronizes the backend-authored plan and uses the Android local-notification library for device scheduling. Existing local schedules are cancelled or replaced when relevant plan data changes. The system also handles missing notification permission through user-facing guidance and supports re-synchronization after relevant application, device, or preference changes.

Table 38: Notification Planning and Delivery Flow

Step	Responsibility
Register device	Associate supported device with authenticated user
Store preferences	Persist category, schedule, frequency, and quiet-hour choices
Generate plan	Backend compiles applicable notification intents/plans
Synchronize	Flutter retrieves the current plan and compares local state
Schedule locally	Android scheduler creates/cancels local reminders
Restore/update	Plan is synchronized again after relevant changes

Table 39: Notification-Flow Responsibilities

Layer	Responsibility
Django Backend	Device registry, preferences, rule/plan generation, synchronization state
Flutter Client	Permission handling, plan synchronization, local-schedule coordination
Android Notification Service	Local delivery at scheduled time

Layer	Responsibility
User Settings	Enable/disable categories, configure timing and quiet hours

6.4.3 UI/UX Refinement

Senior Project 2 included a broad interface refinement rather than isolated cosmetic changes. The Flutter application was organized around feature-based modules with typed models, data-access classes, ChangeNotifier controllers, reusable widgets, and named routes. Shared visual patterns were applied across health modules to improve consistency and reduce navigation effort.

Data-driven screens explicitly represent loading, successful content, validation errors, retry, and empty states. The redesign also simplified high-frequency workflows such as activity tracking, nutrition review, profile management, notifications, and motivation feedback. Four representative screens are shown below; additional operational screens are retained in the user manual in Appendix D.

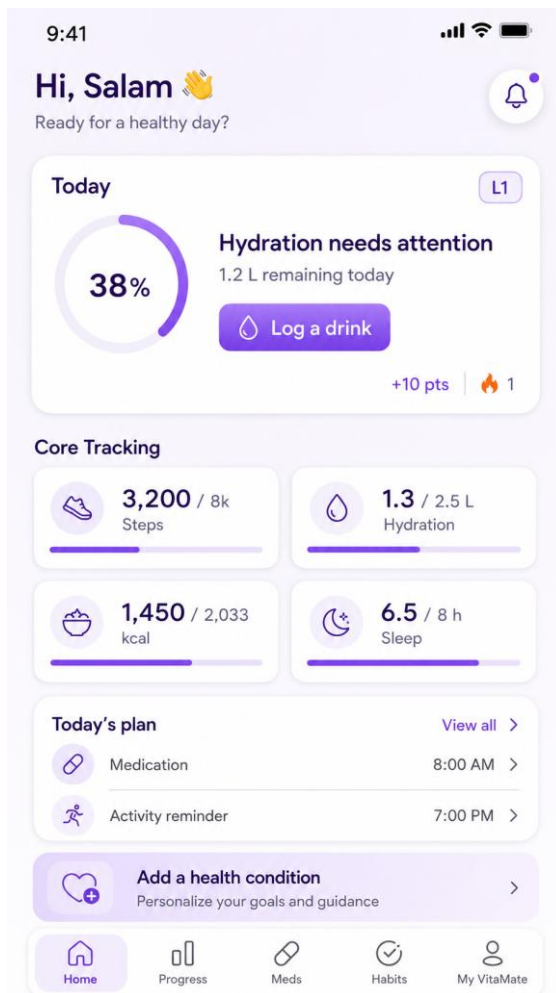


Figure 9: VitaMate Home and Daily Health Overview

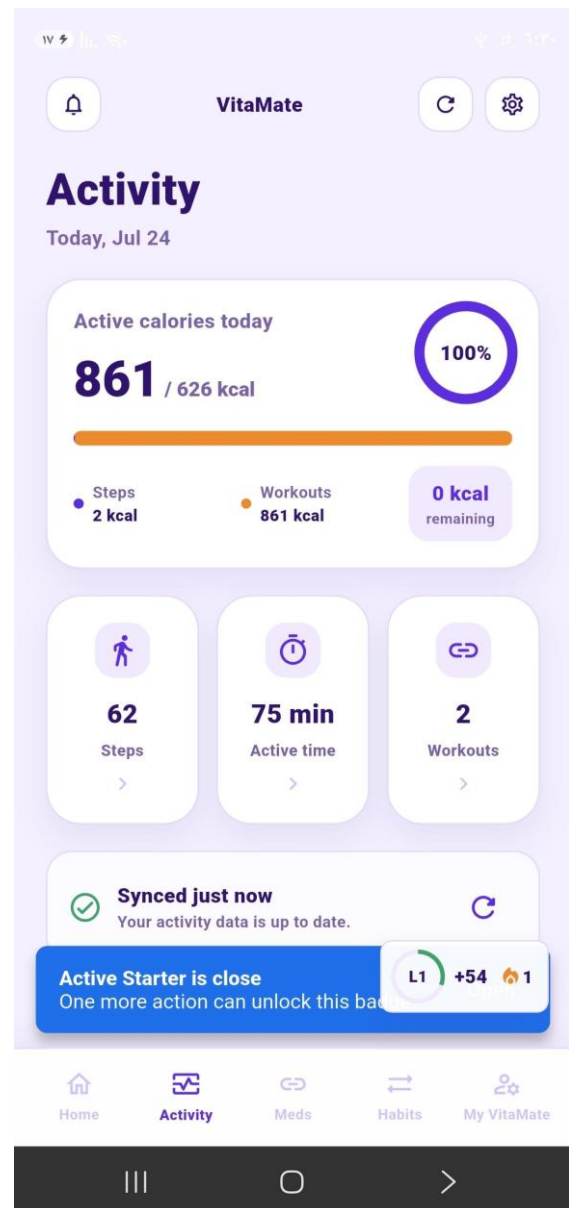


Figure 10: Activity and Steps Interface

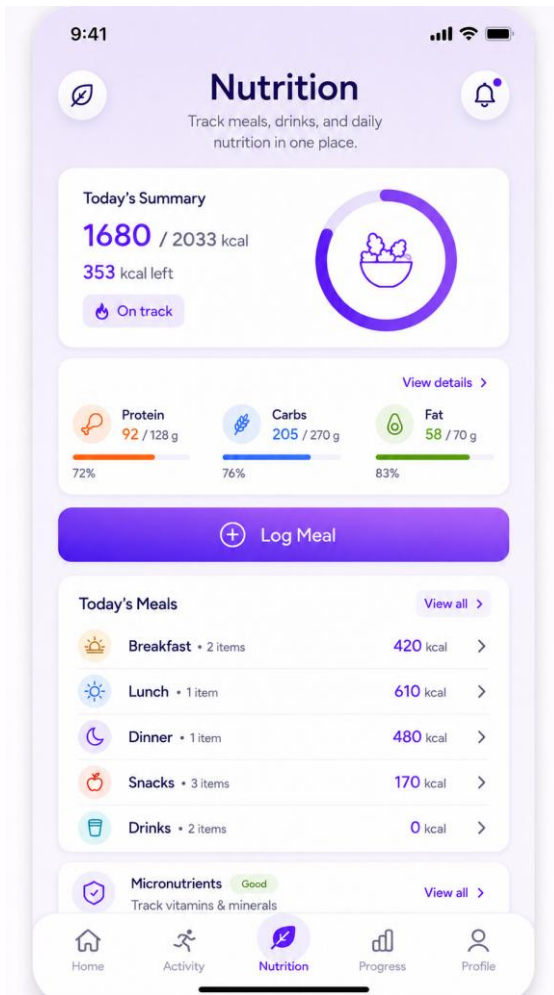


Figure 11: Nutrition Tracking Interface

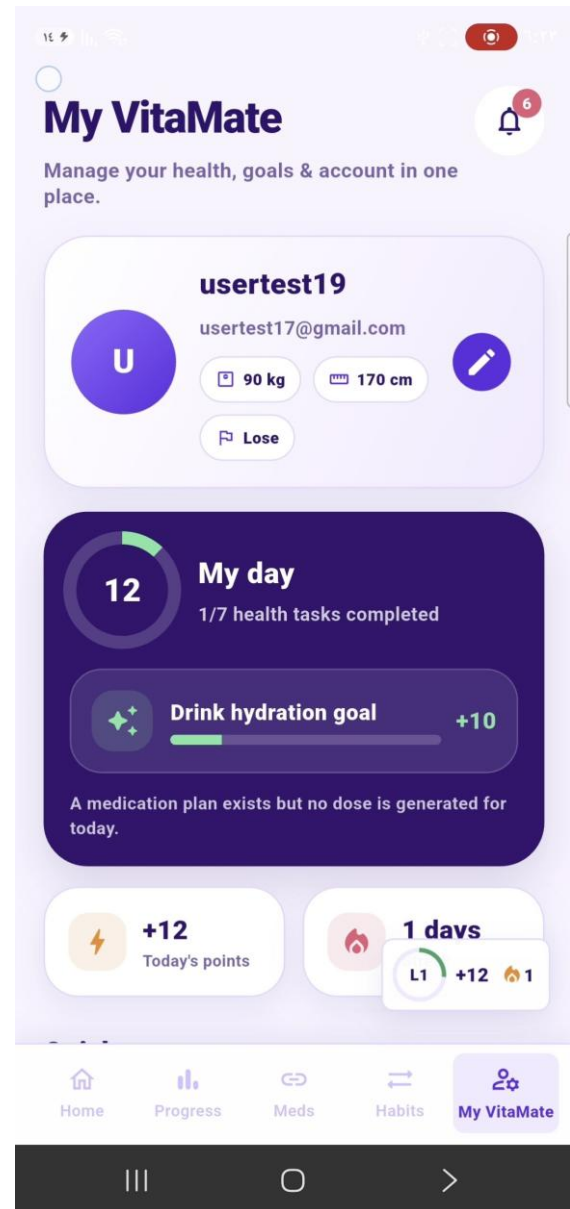


Figure 12: Profile and Self-Management Interface

The final UI presents the project as one coordinated application rather than a collection of unrelated trackers. Automated Flutter tests verify controller behavior, navigation, validation, modal compatibility, compact-screen behavior, and selected visual states; formal external-participant usability evaluation remains future work.

6.4.4 AI Food-Analysis Service Implementation

Meal-image analysis is implemented as a separate FastAPI runtime integrated through Django. The mobile client uploads the image to Django; Django authenticates the user, validates the

request/image, calls the internal AI runtime, and stores the returned result as a provisional analysis session. A provisional result cannot become a final meal until the authenticated user reviews and confirms the suggested food information and quantities.

The implemented pipeline combines food segmentation, dish recognition, optional ingredient evidence, and optional weight estimation. Human-annotated masks were used to select the active segmentation model. Dish recognition operates within the supported catalog. Ingredient and automatic-weight predictions remain supporting suggestions because their evaluated performance did not justify fully authoritative automatic use.

Meal Image → Validation → Food Segmentation → Dish Recognition → Ingredient/Weight Suggestions → User Review & Correction → Canonical Nutrition Finalization

Figure 13: AI Food-Analysis Flow

Table 40: Main AI Pipeline Components

Component	Implementation Role
Food Segmentation	Identifies the food region in the uploaded image
Dish Recognition	Returns ranked dish suggestions from the supported catalog
Ingredient Evidence	Provides optional component suggestions for user review
Weight Estimation	Produces an optional estimate when confidence conditions are satisfied
User Confirmation	Allows correction of dish, ingredients, mappings, and quantities
Nutrition Finalization	Calculates final values from VitaMate’s canonical nutrition catalog

After confirmation, Django maps the reviewed components to canonical nutrition entries and calculates the persistent meal values. The latest evaluated pipeline completed 35/35 release images without runtime failure compared with 16/35 in the oldest baseline, while average analysis time decreased from 123.40 seconds to 6.81 seconds. Detailed evaluation metrics are presented in Chapters 7 and 8.

The second AI service specified for personalized dietary recommendations was not implemented in the submitted release. No recommendation-model code, test results, or performance claims are therefore presented as final implementation evidence.

6.4.5 Cross-Module Integration and Stabilization

Senior Project 2 also corrected integration issues across the existing trackers and strengthened shared coordination. The Health Tracker Coordinator provides unified read-side information for

home, history, and progress views, while the Constraints Layer centralizes effective health restrictions and target adjustments. Profile/settings and unhealthy-habit workflows were also completed and integrated with the relevant tracking, motivation, and notification services.

Table 41: Main Senior Project 2 Additions and Refinements

Area	Final Work
Motivation	Dedicated progress system integrated with verified tracker actions
Notifications	Backend-planned, device-synchronized, locally delivered reminder workflow
UI/UX	Cross-application redesign for clarity, consistency, navigation, and state feedback
AI Food Analysis	Redeveloped review-assisted pipeline integrated with Django and nutrition finalization
Cross-Module Integration	Coordinator/constraints stabilization and defect correction
Validation	Backend, Flutter, integration, security, and AI evaluation evidence

6.5 Implementation Challenges and Solutions

Table 42: Main Implementation Challenges and Applied Solutions

Challenge	Applied Solution	Remaining Consideration
Unstable AI baseline	Improved pipeline stages and runtime handling; retained user review and safe fallbacks	Ingredient/weight automation remains limited by evaluation quality
Unreliable final nutrition from raw AI output	Kept Django/canonical catalog authoritative and required confirmation	Accuracy still depends on catalog coverage and confirmed quantities
Frontend-only reminder design	Introduced backend planning, device registry, preferences, sync, local scheduling, and event state	Delivery still depends on device permission/OS scheduling
Cross-module consistency	Centralized shared services, constraints, and coordinated read models	Regression testing remains important as health rules evolve
Mobile E2E complexity	Used Flutter integration_test with Android emulator and seeded backend	One chronic-condition CI path remained not executed in the final evidence

6.6 Chapter Summary

This chapter documented the verified engineering environment and the final implementation, with emphasis on the four principal Senior Project 2 developments. Motivation was integrated with verified tracker progress; notifications were redesigned as a backend-to-device workflow; the Flutter UI/UX was broadly refined; and the AI meal service was redeveloped into a stable, review-

assisted analyze–confirm–finalize pipeline. Detailed code, configuration, CI, and additional interface evidence are provided in Appendices D and E.

Chapter 7: Testing and Evaluation

7.1 Introduction and Test Strategy

Testing focused on verifiable behavior of the final backend, Flutter client, PostgreSQL-dependent flows, notification integration, and AI meal service. Automated and repeatable evidence was prioritized. A test was accepted when the implemented behavior matched the expected result without an unhandled exception and the corresponding automated execution completed successfully. The detailed test plan, cases, and supporting evidence map are provided in Appendix C.

Table 43: Test Strategy Summary

Area	Main Tools	Primary Objective
Backend	Django TestCase / TransactionTestCase / DRF APITestCase	Services, APIs, permissions, and database behavior
Frontend	flutter test, widget/golden/compact-screen tests	Controllers, navigation, validation, and interface behavior
Integration	Flutter integration_test + Android emulator	Mobile-to-backend workflows
Security	Permission tests + Gitleaks	User isolation and secret exposure
AI	Runtime smoke tests and pipeline evaluation	Service communication, stability, and measured AI behavior
CI	GitHub Actions	Repeat the main quality checks consistently

Table 44: Coverage of Testing Types Required by the Project Guide

Test Type	Status	Evidence / Limitation
Unit / Component	Executed	Django/DRF tests and Flutter unit/widget/controller tests
Integration	Executed	Mobile-to-backend, API/database, notification-plan, and AI integration flows
System	Executed within academic environment	Representative end-to-end user workflows and coordinated read paths
Acceptance	Executed within documented scope	Acceptance criteria summarized in Section 7.7
Performance	Executed for targeted backend endpoints and AI pipeline	Locust dashboard/history evidence plus AI runtime measurements
Load / Production Scale	Not performed	No production-scale concurrency claim is made
Security / Privacy	Executed	JWT access, ownership isolation, permission regression, secret scanning
Usability	Technical UI validation only	Widget/golden/compact-screen tests; no external-participant study
Compatibility	Executed in defined Android environment	Emulator/device-oriented workflow checks
Regression	Executed	Backend and Flutter regression suites

7.2 Test Environment and Scope

Tests were executed across local development and GitHub Actions environments. Backend execution used PostgreSQL-compatible settings and reusable reference data, Flutter integration used an Android emulator with seeded backend data, and the AI service used a fixed release evaluation set for baseline-to-final comparison.

Table 45: Included Test Scope

Scope Area	Coverage
Authentication & Profile	Login/JWT, profile access, user-specific data isolation
Core Trackers	Nutrition, hydration, activity/steps, sleep, medication flows
Conditions & Constraints	Condition data, restrictions, history, reference-data stability
Motivation & Notifications	Motivation behavior; device registration, preferences, and notification-plan integration
AI Meal Analysis	Analyze, provisional review, confirmation/finalization, runtime behavior
Frontend UI	Widgets, navigation, validation, modal compatibility, compact-screen behavior
Integration	Mobile-to-backend smoke workflow, API/database interaction, seeded-user workflow

7.3 Representative Test Cases and Backend Results

Representative cases summarize the strongest directly documented execution evidence rather than duplicating every automated test in the main report.

Table 46: Representative Test Cases and Results

ID	Scenario	Expected Result	Actual Result	Status
TC-01	Valid user login	JWT tokens returned	Authentication succeeded	Passed
TC-02	Cross-user data access	Private records not exposed	Access prevented	Passed
TC-03	Hydration update/delete	Record updates/removes correctly	Current API contract passed	Passed
TC-04	Seven-day progress history	Complete seven-day response	Both history endpoints returned seven days	Passed
TC-05	Expanded backend suite	All selected backend tests pass	240 tests completed with OK	Passed
TC-06	Chronic Flutter E2E in CI	Full chronic flow runs	CI path remained disabled	Not Executed

The final backend suite covered the users, core, gamification, manager, notification_hub, and ai_meals applications. It executed 240 tests in 1110.208 seconds and completed with the result OK. Regression checks also covered corrected progress-history behavior, chronic-condition reference data, nutrition search, seed commands, and user-data isolation.

Table 47: Backend Test Summary

Metric	Result
Executed tests	240
Final status	Passed
Test-runner result	OK
Execution time	1110.208 s
Covered applications	6
Main verified areas	APIs, services, permissions, database, regression

7.4 Frontend and UI/UX Validation

Flutter testing verified controllers, widgets, navigation, form validation, dialogs, loading states, responsive/compact layouts, and updated API contracts. The final Flutter run completed successfully with 53.26% line coverage (11,160 of 20,955 measured lines).

Table 48: Frontend Test Summary

Metric	Result
Test command	flutter test
Final result	All tests passed
Line coverage	53.26%
Covered lines	11,160
Total measured lines	20,955

A formal usability study with external participants was not conducted. Widget, golden, modal-compatibility, and compact-screen tests provide technical UI evidence but are not presented as a substitute for participant-based usability evaluation.

7.5 Integration, Security, and Privacy

Integration testing verified the active mobile-to-backend path using an Android emulator and seeded backend data. Security testing focused on JWT-protected access, ownership isolation, permission regressions, and exposed-secret scanning.

Table 49: Integration and Security Status

Check	Result
Mobile-to-backend smoke flow	Passed
API and database interaction	Passed
Seeded test-user workflow	Passed
Chronic Flutter E2E in CI	Not Executed
JWT-protected API access	Passed
Cross-user data isolation	Passed
Permission regression tests	Passed
Secret scanning	Included in CI
Penetration test / external audit	Not Performed

7.6 Performance and AI Evaluation

Performance evaluation included two complementary forms of evidence. Targeted authenticated backend load tests were used to evaluate the dashboard and history endpoints during optimization, while the AI pipeline was evaluated on the fixed release image set. The targeted backend tests are not presented as production-scale capacity testing.

Table 50: Targeted Backend Performance Optimization Evidence

Endpoint	Metric	Before Optimization	After Optimization	Improvement
/api/dashboard/	Average response time	355.85 ms	142.82 ms	59.87% faster
/api/dashboard/	P95 latency	720 ms	240 ms	66.67% reduction
/api/dashboard/	Requests per second	8.40 RPS	9.12 RPS	8.57% increase
/api/dashboard/	Failures	0	0	No failures
/api/history/	Average response time	5261.65 ms	817.39 ms	84.47% faster
/api/history/	P95 latency	6400 ms	1500 ms	76.56% reduction
/api/history/	Requests per second	2.68 RPS	7.02 RPS	161.94% increase
/api/history/	Failures	0	0	No failures

Table 51: Verified AI Performance Improvement

Metric	Old Baseline	Final Result
Completed analyses	16/35	35/35
Runtime failures	19/35	0/35
Average analysis time	123.40 s	6.81 s

Table 52: Verified Quality and AI Metrics

Metric	Final Result
Backend tests	240 passed / OK
Flutter tests	All passed
Flutter line coverage	53.26% (11,160 / 20,955)
AI pipeline completion	35/35
AI runtime failures	0/35
Ready for user confirmation	32/35
Average AI analysis time	6.81 s
Mean segmentation IoU	90.74%
Dish Top-3 accuracy	57.14%
Finalize mapping success	91.43%

7.7 Acceptance Status and Remaining Validation

The available evidence supports the principal backend, frontend, privacy, integration, and AI-runtime acceptance criteria. Remaining validation items are documented explicitly rather than treated as completed results.

Table 53: Acceptance Status

Acceptance Criterion	Status
Main backend modules pass automated tests	Achieved
Flutter automated tests complete successfully	Achieved
User-owned health data remains isolated	Achieved
Main mobile-to-backend integration path operates	Achieved
AI pipeline completes the release image set	Achieved
Chronic Flutter end-to-end execution in CI	Further Validation
Large-scale backend load evaluation	Future Validation
Formal participant-based usability evaluation	Future Validation
Second AI personalized dietary recommendation service	Not Implemented - Future Work

User confirmation and catalog-based nutrition finalization remain intentional reliability controls for uncertain AI ingredient or weight estimates. Production-scale deployment, broader device/load testing, and participant-based usability evaluation remain appropriate future validation activities.

7.8 Chapter Summary

The final evidence demonstrates stable backend and Flutter automated execution, successful main integration and privacy checks, and major improvement in AI runtime stability and processing time. The report does not overstate areas without sufficient evidence: one chronic-condition CI path was not executed, large-scale load testing was not completed, and no formal external usability or security audit was performed.

Chapter 8: Results and Analysis

8.1 Final System Outcomes

The final VitaMate implementation provides an integrated health and lifestyle platform combining user profiles, core trackers, medication and chronic-condition support, goals/habits, coordinated constraints, motivation, notifications, reports, and review-assisted food analysis. The final result is evaluated according to implemented functions and verified evidence rather than unsupported clinical claims.

Table 54: Final Implemented System Capabilities

Area	Final Capability
Core Platform	Authenticated profile and integrated nutrition, hydration, activity, sleep, medication, condition, goal, and habit modules
Coordination	Constraints Layer and Health Tracker Coordinator for cross-domain targets, warnings, summaries, and progress
Motivation	Points/progress, missions, streaks, achievements/levels, and motivation-feed feedback from verified actions
Notifications	Backend-planned preferences and schedules synchronized to local Android delivery
UI/UX	Consistent Flutter navigation, reusable components, explicit screen states, and refined daily workflows
AI Nutrition	Review-assisted meal-image analysis with provisional results and canonical nutrition finalization
Quality	Automated backend/Flutter tests, integration checks, privacy tests, CI evidence, and AI evaluation

8.2 Senior Project 2 Results

The main Senior Project 2 goal was not to rebuild the original trackers, but to transform the existing baseline into a more integrated, usable, and validated final system. The four principal development areas produced the following outcomes.

Table 55: Main Senior Project 2 Results

Development Area	Final Outcome	Evidence
Motivation System	Integrated verified actions with points/progress, missions, streaks, achievements/levels, and motivational feedback while keeping health score separate.	Implemented backend/frontend modules and automated-suite coverage
Notification System	Replaced the earlier frontend-only approach with device registration, stored preferences, backend plan generation, synchronization, quiet-hour handling, and local Android scheduling.	Notification integration and scheduling architecture; automated/integration coverage
UI/UX Refinement	Redesigned and standardized application interfaces, navigation, reusable	Flutter tests all passed; representative final interfaces

Development Area	Final Outcome	Evidence
	components, and loading/error/empty/success states.	
AI Food Analysis	Redeveloped the pipeline into a stable analyze–review–finalize workflow integrated with Django and the nutrition catalog.	35/35 pipeline completion, 0 runtime failures, 6.81 s average analysis time

8.3 Comparative Analysis with Existing Systems

The comparison emphasizes functional scope rather than claiming superiority over mature commercial products. VitaMate’s contribution is the coordinated combination of several domains within one academic software architecture.

Table 56: Comparative Analysis of VitaMate and Existing Applications

System	Primary Focus	Strength Relative to Scope	Gap / Trade-off
MyFitnessPal	Nutrition & fitness	Strong food and fitness tracking	Less medication/chronic-condition coordination
Samsung Health	General wellness	Broad connected wellness data	No equivalent project-specific constraints/motivation/medication coordination
Medisafe	Medication	Focused scheduling and adherence	Limited multi-domain lifestyle tracking
Google Health	Activity, sleep, connected data	Health-data organization and coaching	Less integration with medication and review-assisted nutrition analysis
VitaMate	Integrated health/lifestyle support	Trackers + medication + conditions + constraints + motivation + notifications + review-assisted AI	Less product maturity and external validation than established applications

8.4 Achievement of Project Objectives

The eight specific objectives defined in Chapter 1 were assessed directly against the final implementation and verified test evidence.

Table 57: Achievement of Project Objectives

Project Objective	Evidence of Achievement	Status
1. Develop secure user-account and health-profile management for personalized tracking.	Authentication/profile APIs, secure sessions, ownership isolation, editable health profile.	Achieved
2. Integrate nutrition, hydration, activity, sleep, medication, chronic-condition, goal, and habit-management functions within one mobile platform.	All listed domains are available through the Flutter client and Django backend.	Achieved

Project Objective	Evidence of Achievement	Status
3. Coordinate health-related targets, restrictions, and tracker behavior through a centralized constraints mechanism.	Constraints Layer and coordinated read models apply shared restrictions/targets.	Achieved
4. Provide progress dashboards, reports, and cross-domain health summaries based on recorded user data.	Home/progress summaries, history, indicators, and coordinated health views are implemented.	Achieved
5. Develop a motivation system that links verified user actions with points, missions, streaks, achievements, and motivational feedback.	Dedicated motivation module integrated after verified tracker processing.	Achieved
6. Develop a coordinated notification system that supports user preferences, reminder planning, synchronization, and local device delivery.	Backend device/preferences/plan workflow synchronized to Flutter local scheduling.	Achieved
7. Provide review-assisted AI food-image analysis integrated with the nutrition workflow.	Analyze–review–finalize flow, canonical catalog finalization, and user confirmation.	Achieved with User Review
8. Improve system usability and evaluate the final implementation through functional, integration, security, performance, interface, and AI-related testing.	UI refinement plus backend, Flutter, integration, privacy, and AI evidence; no formal external usability study.	Achieved within Project Evaluation Scope

8.5 Key Performance Indicators and AI Results

The final indicators combine software-quality evidence with the directly measured AI release results. AI accuracy measures are interpreted as support for a review-assisted workflow rather than evidence for unrestricted automatic nutrition assessment.

Table 58: Final Key Performance Indicators

KPI	Final Result
Backend automated tests	240 passed; suite result OK
Flutter automated tests	All tests passed
Flutter line coverage	53.26% (11,160 / 20,955)
Main mobile-to-backend smoke flow	Passed
Cross-user data isolation	Passed
AI pipeline completion	100% (35/35)
AI runtime failure rate	0% (0/35)
Ready for user confirmation	32/35
Average AI analysis time	6.81 s
Mean segmentation IoU	90.74%
Dish Top-3 accuracy	57.14%
Finalize mapping success	91.43%

The strongest AI improvement was operational stability: the oldest baseline completed only 16 of 35 images with an average analysis time of 123.40 seconds, while the final pipeline completed all 35 images without runtime failure at 6.81 seconds average. Ingredient recognition and automatic weight estimation were not accepted as fully authoritative outputs, so catalog-based review and manual correction remain part of the final workflow.

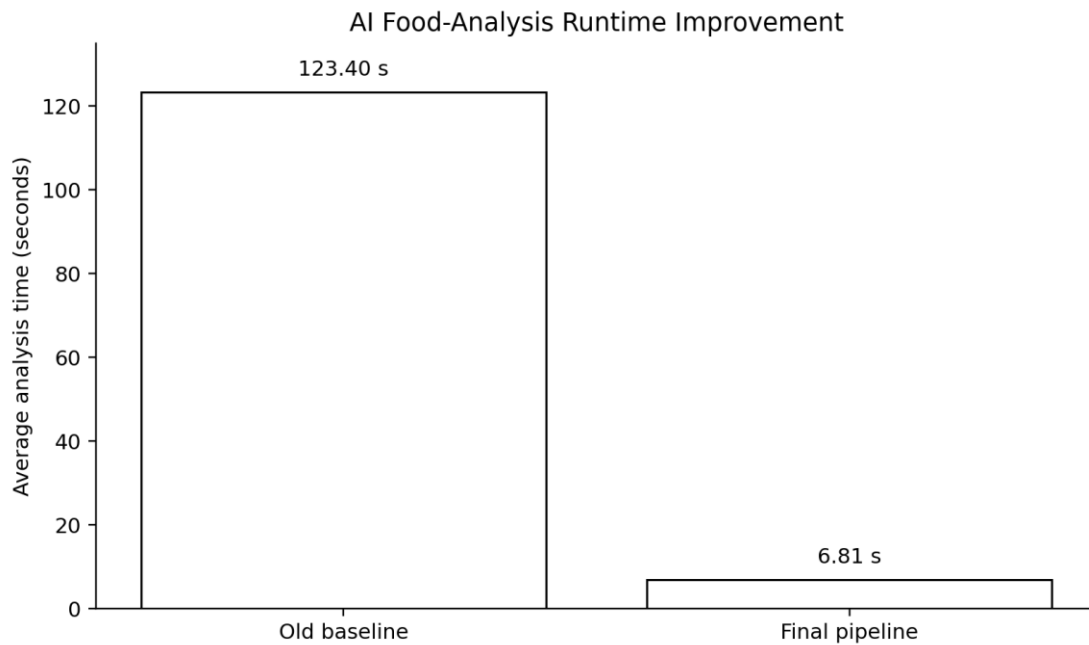


Figure 14: AI Food-Analysis Average Runtime: Baseline vs Final Pipeline

8.6 Strengths and Limitations

Table 59: Main Strengths and Limitations

Strengths	Limitations
Integrates multiple health/lifestyle domains in one application	No clinical validation; not a diagnostic or treatment system
Coordinates conditions, goals, medications, and trackers through shared constraints	No formal usability study with external participants
Uses a layered, testable backend with automated evidence	Large-scale backend load/production testing not completed
Provides backend-planned notifications with local Android delivery	One chronic-condition Flutter E2E path remained not executed in CI
Uses user confirmation before uncertain AI results become final records	AI evaluation sets remain limited; ingredient/weight automation is not fully autonomous
Achieved stable AI execution and large runtime improvement	Production deployment evidence is outside the submitted validation
Final report distinguishes implemented AI food analysis from planned intelligent services	Second AI personalized dietary recommendation service was deferred and not implemented

These limitations mainly affect the scale of validation and product maturity rather than the existence of the core implemented functions. Broader clinical, participant, security, device/load, deployment, and AI evaluation would be required before presenting VitaMate as a mature health product.

8.7 Chapter Summary

The final results show that VitaMate achieved its principal software-engineering objectives and, in Senior Project 2, completed the motivation system, coordinated notification workflow, UI/UX refinement, AI-service redevelopment, integration, and final evaluation. Automated testing confirmed stable backend and Flutter execution, while the AI pipeline improved substantially in runtime stability and speed. Remaining validation limits are documented explicitly and define realistic future work rather than hidden incomplete scope.

Conclusion

VitaMate was developed as an integrated mobile platform for health and lifestyle self-monitoring. The final system combined profile management, nutrition, hydration, activity and steps, sleep, medication support, chronic-condition data and constraints, goals, unhealthy-habit management, vitamins and minerals, reports, motivation, coordinated notifications, and review-assisted AI food analysis. The project contribution was primarily architectural and integrative: several health-related domains were coordinated through shared backend services while health-support outputs remained non-diagnostic and under user control.

Main Achievements

Senior Project 2 transformed the Senior Project 1 baseline into a more integrated and validated final system. The principal achievements were the dedicated motivation subsystem linked to verified actions; the backend-planned, device-synchronized notification workflow; broad UI/UX refinement across high-frequency workflows; and redevelopment of the AI meal-analysis pipeline into an analyze-review-finalize process. Verification included 240 successful backend tests, successful Flutter execution with 53.26% line coverage, mobile-to-backend integration checks, privacy and permission tests, targeted backend performance optimization, and AI evaluation on the fixed release set. The final AI pipeline completed 35/35 images without runtime failure and reduced average analysis time to 6.81 seconds.

Challenges and Lessons Learned

The main technical challenges were cross-module consistency, notification synchronization, reliable finalization of uncertain AI output, and maintaining repeatable mobile integration tests. These challenges reinforced the value of centralized domain services, idempotent state changes, user confirmation for uncertain intelligent outputs, incremental integration, and explicit regression evidence. Time and team-size constraints also required strict prioritization: completed work was documented with evidence, while unvalidated or deferred work was not presented as a finished result.

Recommendations and Future Work

Future work should first implement and independently evaluate the second AI service for personalized dietary recommendations (SRS FR-60-FR-66), using verified profile, nutrition, activity, sleep, hydration, and chronic-condition constraints while preserving user review and health-safety boundaries. Additional work should include larger and more diverse AI evaluation datasets, participant-based usability testing, production-scale load and security testing, wider Android/device compatibility validation, deployment hardening, and external clinical or domain review before any stronger health-effectiveness claims are considered.

References

- [1] World Health Organization, Classification of Digital Interventions, Services and Applications in Health: A Shared Language to Describe the Uses of Digital Technology for Health, 2nd ed. Geneva, Switzerland: World Health Organization, 2023.
- [2] R. Berry, A. Kassavou, and S. Sutton, “Does self-monitoring diet and physical activity behaviors using digital technology support adults with obesity or overweight to lose weight? A systematic literature review with meta-analysis,” *Obesity Reviews*, vol. 22, no. 10, Art. no. e13306, Oct. 2021, doi: 10.1111/obr.13306.
- [3] M. Aguiar, M. Trujillo, D. Chaves, R. Álvarez, and G. Epelde, “mHealth apps using behavior change techniques to self-report data: Systematic review,” *JMIR mHealth and uHealth*, vol. 10, no. 9, Art. no. e33247, Sep. 2022, doi: 10.2196/33247.
- [4] R. Kaur, R. Kumar, and M. Gupta, “Deep neural network for food image classification and nutrient identification: A systematic review,” *Reviews in Endocrine and Metabolic Disorders*, vol. 24, no. 4, pp. 633–653, Aug. 2023, doi: 10.1007/s11154-023-09795-4.

- [5] World Health Organization, “Digital health research: Background,” Global Observatory on Health Research and Development. [Online]. Available: <https://www.who.int/observatories/global-observatory-on-health-research-and-development/analyses-and-syntheses/digital-health/background>. [Accessed: Jul. 31, 2026].
- [6] H. L. Tong, J. C. Quiroz, A. B. Kocaballi, S. Chan Moi Fat, K. P. Dao, H. Gehringer, C. K. Chow, and L. Laranjo, “Personalized mobile technologies for lifestyle behavior change: A systematic review, meta-analysis, and meta-regression,” *Preventive Medicine*, vol. 148, Art. no. 106532, Jul. 2021, doi: 10.1016/j.ypmed.2021.106532.
- [7] R. Jakob, S. Harperink, A. M. Rudolf, E. Fleisch, S. Haug, J. L. Mair, A. Salamanca-Sanabria, and T. Kowatsch, “Factors influencing adherence to mHealth apps for prevention or management of noncommunicable diseases: Systematic review,” *Journal of Medical Internet Research*, vol. 24, no. 5, Art. no. e35371, May 2022, doi: 10.2196/35371.
- [8] A. M. Belete, B. N. Gemed, T. Y. Akalu, Y. A. Aynalem, and W. S. Shiferaw, “What is the effect of mobile phone text message reminders on medication adherence among adult type 2 diabetes mellitus patients: A systematic review and meta-analysis of randomized controlled trials,” *BMC Endocrine Disorders*, vol. 23, no. 1, Art. no. 18, Jan. 2023, doi: 10.1186/s12902-023-01268-8.

- [9] M. Arshed, A. B. Mahmud, H. S. Minhat, L. P. Ying, and M. F. Umer, “Effectiveness of mHealth interventions in medication adherence among patients with cardiovascular diseases: A systematic review,” *Diseases*, vol. 11, no. 1, Art. no. 41, Mar. 2023, doi: 10.3390/diseases11010041.
- [10] M. Milne-Ives, S. R. Homer, J. Andrade, and E. Meinert, “Potential associations between behavior change techniques and engagement with mobile health apps: A systematic review,” *Frontiers in Psychology*, vol. 14, Art. no. 1227443, Sep. 2023, doi: 10.3389/fpsyg.2023.1227443.
- [11] MyFitnessPal, “Calorie tracker and BMR calculator to reach your goals.” [Online]. Available: <https://www.myfitnesspal.com/>. [Accessed: Jul. 31, 2026].
- [12] Samsung Electronics, “Samsung Health.” [Online]. Available: <https://www.samsung.com/us/apps/samsung-health/>. [Accessed: Jul. 31, 2026].
- [13] Medisafe, “Download the app | Medisafe medication management.” [Online]. Available: <https://medisafe.com/download-the-app>. [Accessed: Jul. 31, 2026].

[14] Google, “Explore the Google Health app,” Google Health Help.

[Online]. Available:

<https://support.google.com/googlehealth/answer/14237011?hl=en>.

[Accessed: Jul. 31, 2026].

Appendices

The appendices retain detailed requirements, UML, testing, user-operation, and technical evidence that supports the main report without interrupting its flow.

Appendix A: Detailed Use Case Specifications

The following tables expand the seven high-level use cases summarized in Chapter 4. They document the principal trigger, conditions, inputs/outputs, main flow, and alternative behavior without introducing implementation-level code details.

UC-01 — Manage Account and Health Profile

Table 60: Detailed Use Case Specification - UC-01: Manage Account and Health Profile

Use Case ID	UC-01
Use Case Name	Manage Account and Health Profile
Primary Actor	User
Trigger	Registration, sign-in, profile edit, or account-management action
Preconditions	System available; authentication required for protected profile actions
Inputs	Credentials; personal/health data; preferences; optional profile image
Outputs	Authentication/profile result and recalculated personalized values where applicable
Main Flow	1) Open account/profile function. 2) Select operation. 3) Enter/update data. 4) Validate. 5) Store approved changes. 6) Display updated state.
Alternative / Exception Flow	Invalid/duplicate data is rejected; expired session requires sign-in; account deletion requires confirmation.
Postconditions	Account/session or approved profile changes are stored.

UC-02 — Manage Daily Health Tracking

Table 61: Detailed Use Case Specification - UC-02: Manage Daily Health Tracking

Use Case ID	UC-02
Use Case Name	Manage Daily Health Tracking
Primary Actor	User
Trigger	Open tracker, record data, start activity, or respond to reminder
Preconditions	Authenticated; required device permission granted for automatic step data

Use Case ID	UC-02
Inputs	Steps, activity details, water/beverage amount, sleep times, targets
Outputs	Updated records, progress, history, and validation feedback
Main Flow	1) Select tracker. 2) Review current state. 3) Record/start supported action. 4) Validate/store. 5) Recalculate affected progress. 6) Display result.
Alternative / Exception Flow	Permission denied disables automatic retrieval; invalid data is rejected; failure allows retry without duplication.
Postconditions	Valid record and affected summaries are updated.

UC-03 — Manage Nutrition and AI-Assisted Meals

Table 62: Detailed Use Case Specification - UC-03: Manage Nutrition and AI-Assisted Meals

Use Case ID	UC-03
Use Case Name	Manage Nutrition and AI-Assisted Meals
Primary Actor	User
Trigger	Add meal manually or submit meal image
Preconditions	Authenticated; valid data/image for selected path
Inputs	Meal data or image; user corrections/confirmation
Outputs	Meal details, provisional AI result, nutrition values, warnings
Main Flow	1) Choose manual or AI path. 2) Validate input/image. 3) Calculate/estimate result. 4) Review AI result when used. 5) Confirm. 6) Finalize and update totals.
Alternative / Exception Flow	Invalid/low-confidence results show guidance; user may edit/reject; service failure does not create a final meal.
Postconditions	Only confirmed meal data is persisted and included in nutrition totals.

UC-04 — Manage Medications and Chronic Conditions

Table 63: Detailed Use Case Specification - UC-04: Manage Medications and Chronic Conditions

Use Case ID	UC-04
Use Case Name	Manage Medications and Chronic Conditions
Primary Actor	User
Trigger	Add/update medication, condition, reading, or dose status
Preconditions	Authenticated
Inputs	Medication plan/schedule, dose action, condition/readings
Outputs	Schedules, adherence, condition summary, restrictions/warnings
Main Flow	1) Open module. 2) Add/update data. 3) Validate/store. 4) Update reminders/adherence/constraints. 5) Display state.
Alternative / Exception Flow	Invalid data rejected; dose can be taken/skipped/missed; conflicts generate warning without modifying treatment.

Use Case ID	UC-04
Postconditions	Valid records, schedules, and applicable constraints are updated.

UC-05 — Manage Goals and Unhealthy Habits

Table 64: Detailed Use Case Specification - UC-05: Manage Goals and Unhealthy Habits

Use Case ID	UC-05
Use Case Name	Manage Goals and Unhealthy Habits
Primary Actor	User
Trigger	Create/update goal or habit plan; record progress/relapse
Preconditions	Authenticated
Inputs	Goal/habit type, target, duration, progress/adherence
Outputs	Status, progress, streaks, motivational feedback
Main Flow	1) Create/select plan. 2) Enter details. 3) Validate/store. 4) Record progress/adherence. 5) Update related tracking/motivation. 6) Display result.
Alternative / Exception Flow	Plan may be paused/resumed/completed/cancelled; relapse preserves previous progress; invalid targets rejected.
Postconditions	Plan and progress history remain stored consistently.

UC-06 — Manage Notifications and Motivational Support

Table 65: Detailed Use Case Specification - UC-06: Manage Notifications and Motivational Support

Use Case ID	UC-06
Use Case Name	Manage Notifications and Motivational Support
Primary Actor	User
Trigger	Update notification preferences; complete eligible action; reminder becomes due
Preconditions	Authenticated; notification permission required for local delivery
Inputs	Preferences, schedules, quiet hours, verified tracker actions
Outputs	Notification plan/status; points, missions, streaks, achievements/feed feedback
Main Flow	1) Configure preferences. 2) Backend stores/evaluates plan. 3) Flutter synchronizes plan. 4) Device schedules reminders. 5) Verified actions update motivation. 6) Display status/feedback.
Alternative / Exception Flow	Permission denial shows guidance; quiet hours affect non-priority delivery; duplicate events do not duplicate rewards.
Postconditions	Preferences/plan and applicable motivational state are updated.

UC-07 — View Reports and Supportive Guidance

Table 66: Detailed Use Case Specification - UC-07: View Reports and Supportive Guidance

Use Case ID	UC-07
Use Case Name	View Reports and Supportive Guidance
Primary Actor	User
Trigger	Open home, history, report, or progress view
Preconditions	Authenticated; relevant data required for detailed trends
Inputs	Profile, tracker records, goals, conditions, medications, constraints
Outputs	Dashboard summaries, trends, warnings, supportive guidance
Main Flow	1) Select report/view. 2) Retrieve relevant records. 3) Calculate summaries/trends. 4) Apply applicable constraints. 5) Display coordinated result.
Alternative / Exception Flow	Missing data produces limited summary; loading failure allows retry; supportive outputs remain non-diagnostic.
Postconditions	Requested information is displayed without automatic treatment changes.

Appendix B: Additional UML and Agile Artifacts

These artifacts are valid supporting models but were moved out of the main chapter to reduce repetition and keep the report focused on the principal Senior Project 2 workflows.

B.1 Additional Activity Diagram

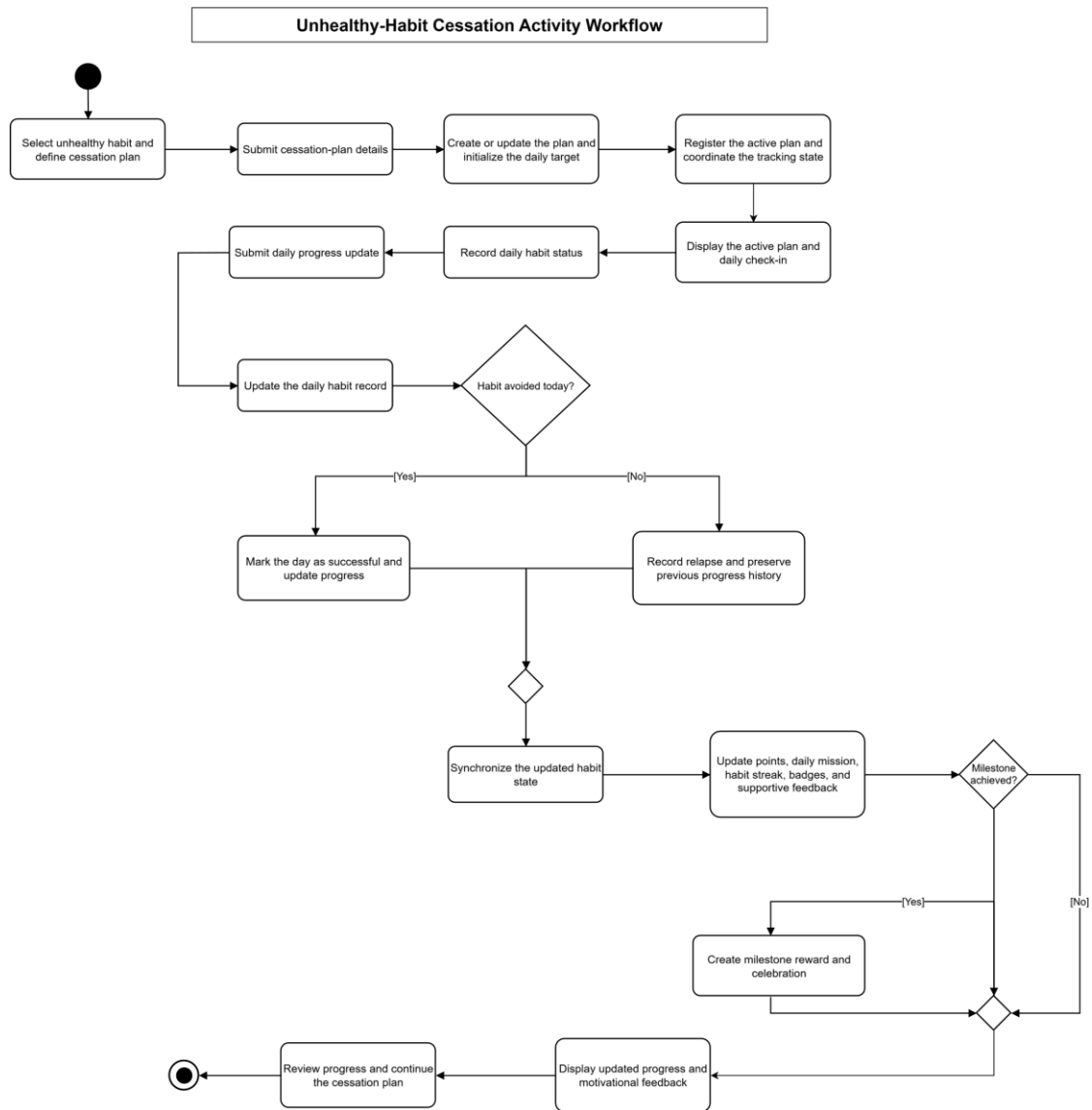


Figure 15: Activity Diagram for Unhealthy-Habit Cessation

B.2 Additional Sequence Diagram

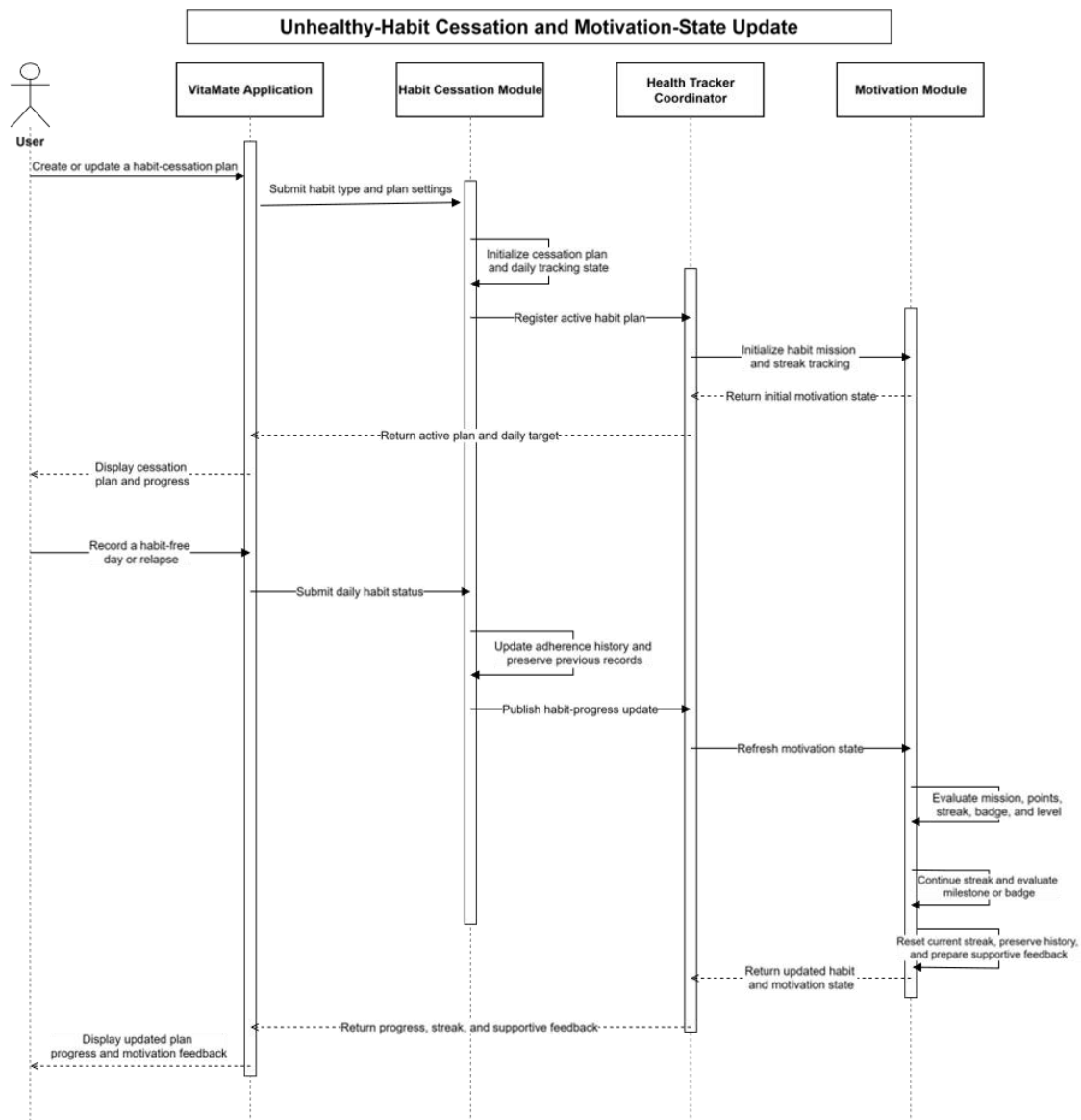
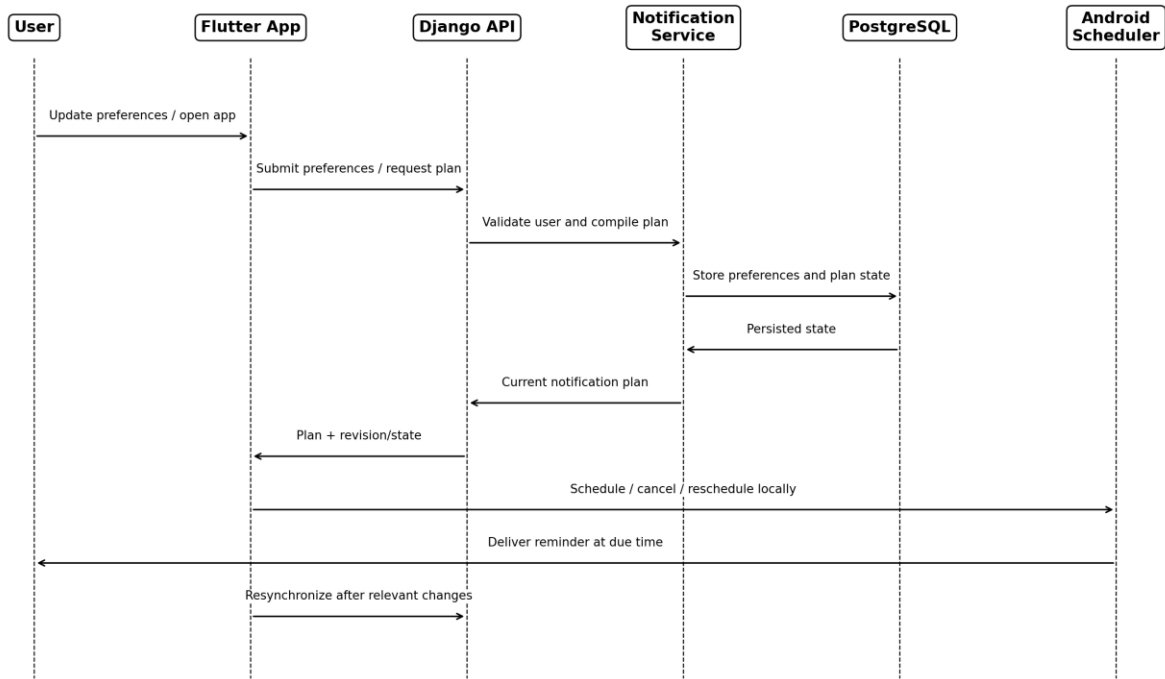


Figure 16: Sequence Diagram for Unhealthy-Habit Cessation and Motivation Update

B.3 Notification Planning and Delivery Sequence Diagram



Backend planning remains authoritative for preferences and plan generation; delivery is performed locally on Android.

Figure 17: Sequence Diagram for Notification Planning, Synchronization, and Local Delivery

B.4 Detailed Agile Backlog

Table 67: Detailed Senior Project 2 Backlog and Acceptance Focus

ID	Work Area	Acceptance Focus
PB-01	Core integration	Main modules communicate through stable APIs and coordinated data views
PB-02	Constraints coordination	Applicable targets/restrictions update consistently across affected views
PB-03	Motivation	Verified actions update motivation without duplicate rewards
PB-04	Notifications	Preferences and plans synchronize to supported local scheduling
PB-05	UI/UX	Core screens use consistent navigation and explicit interface states
PB-06	AI food analysis	Provisional analysis supports review and canonical finalization
PB-07	Testing/stabilization	Final automated suites and integration evidence complete within documented scope
PB-08	Documentation/delivery	Report and presentation align with final implemented scope

Table 68: Final Sprint Objectives and Deliverables

Sprint	Objective	Delivered Focus
Sprint 1	Planning and integration baseline	Scope refinement, dependency review, integration preparation
Sprint 2	Core development	Motivation and notification development; selected cross-module integration
Sprint 3	AI and UI/UX refinement	AI food-analysis development/integration; interface redesign and workflow refinement
Sprint 4	System integration and stabilization	Cross-module coordination; notification/motivation integration; defect correction
Sprint 5	Final validation and delivery	Testing, stabilization, documentation, final demonstration, and delivery

Appendix C: Detailed Test Plan, Traceability, and Evidence

This appendix consolidates the detailed evidence map supporting Chapters 4 and 7. It does not fabricate test outputs that were not captured; items without execution evidence remain explicitly identified.

C.1 Requirement-Level Traceability

The approved SRS defines 96 atomic functional requirements. The final RTM below preserves those SRS identifiers while mapping them to the consolidated use cases, implementation components, and verification areas used in this final report. In accordance with the submitted implementation, all SRS functional groups are treated as completed except FR-60-FR-66, which define the second AI personalized dietary recommendation service and remain deferred future work.

Table 69: Final SRS-Level Requirements Traceability Matrix

Req. ID	Area	UC	Component	Verification	Status	Priority
FR-01	Profile Management	UC-01	Authentication / Profile	API, security, profile and ownership tests	Completed	Must
FR-02	Profile Management	UC-01	Authentication / Profile	API, security, profile and ownership tests	Completed	Must
FR-03	Profile Management	UC-01	Authentication / Profile	API, security, profile and ownership tests	Completed	Must
FR-04	Profile Management	UC-01	Authentication / Profile	API, security, profile and ownership tests	Completed	Must
FR-05	Profile Management	UC-01	Authentication / Profile	API, security, profile and ownership tests	Completed	Must
FR-06	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-07	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-08	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-09	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must

Req. ID	Area	UC	Component	Verification	Status	Priority
FR-10	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-11	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-12	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-13	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-14	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-15	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-16	Physical Activity Tracking	UC-02	Activity / Device Services	Functional / device / integration / regression	Completed	Must
FR-17	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-18	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-19	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-20	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-21	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-22	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-23	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-24	Nutrition Tracking	UC-03	Nutrition	Functional / integration / regression	Completed	Must
FR-25	Hydration Management	UC-02	Hydration	Functional / notification / regression	Completed	Must
FR-26	Hydration Management	UC-02	Hydration	Functional / notification / regression	Completed	Must
FR-27	Hydration Management	UC-02	Hydration	Functional / notification / regression	Completed	Must

Req. ID	Area	UC	Component	Verification	Status	Priority
FR-28	Hydration Management	UC-02	Hydration	Functional / notification / regression	Completed	Must
FR-29	Hydration Management	UC-02	Hydration	Functional / notification / regression	Completed	Must
FR-30	Sleep Tracking	UC-02	Sleep	Functional / scheduling / regression	Completed	Must
FR-31	Sleep Tracking	UC-02	Sleep	Functional / scheduling / regression	Completed	Must
FR-32	Sleep Tracking	UC-02	Sleep	Functional / scheduling / regression	Completed	Must
FR-33	Sleep Tracking	UC-02	Sleep	Functional / scheduling / regression	Completed	Must
FR-34	Motivation and Reporting	UC-06 / UC-07	Motivation / Reporting	Motivation / reporting / integration	Completed	Must
FR-35	Motivation and Reporting	UC-06 / UC-07	Motivation / Reporting	Motivation / reporting / integration	Completed	Must
FR-36	Motivation and Reporting	UC-06 / UC-07	Motivation / Reporting	Motivation / reporting / integration	Completed	Must
FR-37	Personal Goals Management	UC-05	Goals	Functional / progress	Completed	Must
FR-38	Personal Goals Management	UC-05	Goals	Functional / progress	Completed	Must
FR-39	Personal Goals Management	UC-05	Goals	Functional / progress	Completed	Must
FR-40	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling / adherence / integration	Completed	Must
FR-41	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling / adherence / integration	Completed	Must
FR-42	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling / adherence / integration	Completed	Must
FR-43	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling / adherence / integration	Completed	Must
FR-44	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling / adherence / integration	Completed	Must
FR-45	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling	Completed	Should

Req. ID	Area	UC	Component	Verification	Status	Priority
				adherence / integration		
FR-46	Medication Management	UC-04	Medication / Notification Hub	Functional / scheduling / adherence / integration	Completed	Should
FR-47	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Must
FR-48	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Should
FR-49	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Must
FR-50	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Must
FR-51	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Must
FR-52	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Should
FR-53	Unhealthy Habit Management	UC-05	Habits / Motivation	Functional / motivation / progress	Completed	Should
FR-54	AI-Based Food Analysis	UC-03	AI Meals / Nutrition	AI contract / pipeline / integration / performance	Completed - User Review Required	Should
FR-55	AI-Based Food Analysis	UC-03	AI Meals / Nutrition	AI contract / pipeline / integration / performance	Completed - User Review Required	Should
FR-56	AI-Based Food Analysis	UC-03	AI Meals / Nutrition	AI contract / pipeline / integration / performance	Completed - User Review Required	Should
FR-57	AI-Based Food Analysis	UC-03	AI Meals / Nutrition	AI contract / pipeline / integration / performance	Completed - User Review Required	Should
FR-58	AI-Based Food Analysis	UC-03	AI Meals / Nutrition	AI contract / pipeline / integration / performance	Completed - User Review Required	Should
FR-59	AI-Based Food Analysis	UC-03	AI Meals / Nutrition	AI contract / pipeline / integration / performance	Completed - User Review Required	Should
FR-60	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Must

Req. ID	Area	UC	Component	Verification	Status	Priority
FR-61	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Must
FR-62	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Must
FR-63	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Must
FR-64	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Should
FR-65	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Must
FR-66	Personalized Diet Recommendation AI	UC-03 / UC-07	Planned Recommendation Service	Not executed	Deferred - Not Implemented	Should
FR-67	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-68	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-69	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Could
FR-70	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-71	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-72	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-73	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-74	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must

Req. ID	Area	UC	Component	Verification	Status	Priority
FR-75	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-76	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Should
FR-77	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-78	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Should
FR-79	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Must
FR-80	Chronic Condition Support	UC-04 / UC-07	Conditions / Constraints / Coordinator	Functional / constraints / integration / regression	Completed	Could
FR-81	Beverage Tracking	UC-02 / UC-03	Hydration / Nutrition	Functional / nutrition / hydration / integration	Completed	Must
FR-82	Beverage Tracking	UC-02 / UC-03	Hydration / Nutrition	Functional / nutrition / hydration / integration	Completed	Must
FR-83	Beverage Tracking	UC-02 / UC-03	Hydration / Nutrition	Functional / nutrition / hydration / integration	Completed	Must
FR-84	Beverage Tracking	UC-02 / UC-03	Hydration / Nutrition	Functional / nutrition / hydration / integration	Completed	Should
FR-85	Beverage Tracking	UC-02 / UC-03	Hydration / Nutrition	Functional / nutrition / hydration / integration	Completed	Must
FR-86	Beverage Tracking	UC-02 / UC-03	Hydration / Nutrition	Functional / nutrition / hydration / integration	Completed	Must
FR-87	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must
FR-88	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must
FR-89	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must
FR-90	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must

Req. ID	Area	UC	Component	Verification	Status	Priority
FR-91	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must
FR-92	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must
FR-93	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Should
FR-94	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Should
FR-95	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Must
FR-96	Vitamins and Minerals Monitoring	UC-03 / UC-07	Nutrition / Reports / Constraints	Functional / reporting / integration	Completed	Should

C.2 Executed Representative Test Cases

Table 70: Detailed Representative Executed Tests

ID	Scenario	Expected	Actual	Status
TC-01	Valid user login	JWT tokens returned	Authentication succeeded	Passed
TC-02	Cross-user data access	Private records not exposed	Access prevented	Passed
TC-03	Hydration update/delete	Record updates/removes correctly	Current API contract passed	Passed
TC-04	Seven-day progress history	Seven-day response returned	Both history endpoints returned seven days	Passed
TC-05	Expanded backend test suite	All selected backend tests pass	240 tests completed with OK	Passed
TC-06	Chronic Flutter E2E in CI	Full chronic flow runs	CI path remained disabled	Not Executed
TC-07	Personalized dietary recommendation AI service	Generate a personalized recommendation	Not executed because the service was deferred	Not Implemented

C.3 Evidence Summary

Table 71: Final Test Evidence Map

Evidence Area	Verified Evidence
Backend	240 tests; result OK; 1110.208 s; APIs/services/permissions/database/regression

Evidence Area	Verified Evidence
Frontend	All Flutter tests passed; 53.26% coverage; 11,160 / 20,955 lines
Integration	Mobile-backend smoke, API/database, and seeded-user workflows passed
Security/Privacy	JWT protection, cross-user isolation, permission regressions passed; secret scanning in CI
AI	35/35 completion, 0 failures, 6.81 s average, 90.74% segmentation IoU, 57.14% dish Top-3, 91.43% finalize mapping
Not Completed as Formal Evidence	Second AI personalized dietary recommendation service; production-scale load test; external usability study; penetration test/external security audit

Appendix D: User Manual

D.1 Overview and Operating Requirements

VitaMate is an Android mobile application for integrated lifestyle and health self-monitoring. Most application data is synchronized through the Django backend and therefore requires the configured backend/database environment and network connectivity. Some device functions additionally require notification, activity/step, or camera/media permissions depending on the selected workflow.

VitaMate does not diagnose disease, prescribe treatment, or independently modify medication. AI meal results are assistive estimates and must be reviewed before finalization.

D.2 Installation and Setup

VitaMate is installed as an Android application. The configured Django backend and PostgreSQL database must be available for synchronized application data. The FastAPI AI runtime is additionally required only when the AI-assisted meal workflow is used. Depending on the selected feature, Android notification, activity/step, and camera/media permissions must be granted. Development or academic deployment configuration should be completed using the repository environment files and documented dependency settings; credentials must not be embedded in the application package.

D.3 Quick Start

1. Sign in or create an account and complete the health profile.
2. Use the Home screen to open the required tracker or management module.
3. Record daily nutrition, hydration, activity, sleep, medication, goal, habit, or related health data as needed.
4. Configure notification preferences and review motivational progress and reports.
5. When using AI-assisted meal analysis, submit a supported image, review the provisional result, correct it when necessary, and confirm only the intended final meal.

D.4 Main Interfaces

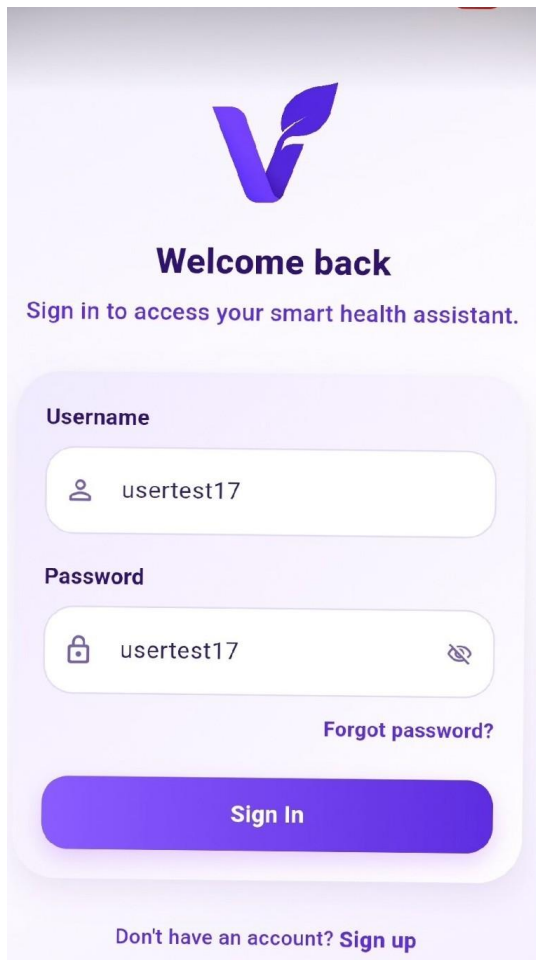


Figure 18: VitaMate Login Interface

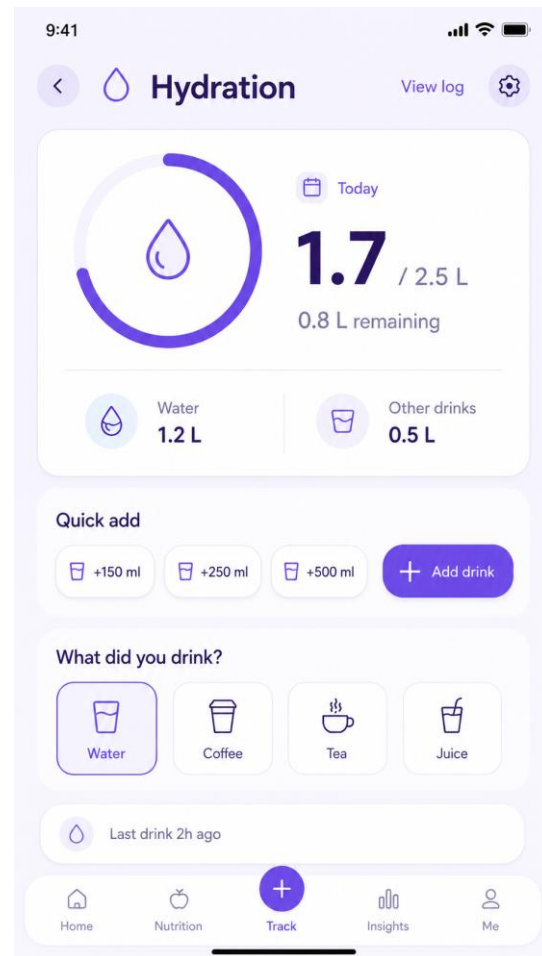


Figure 19: Hydration Tracking Interface

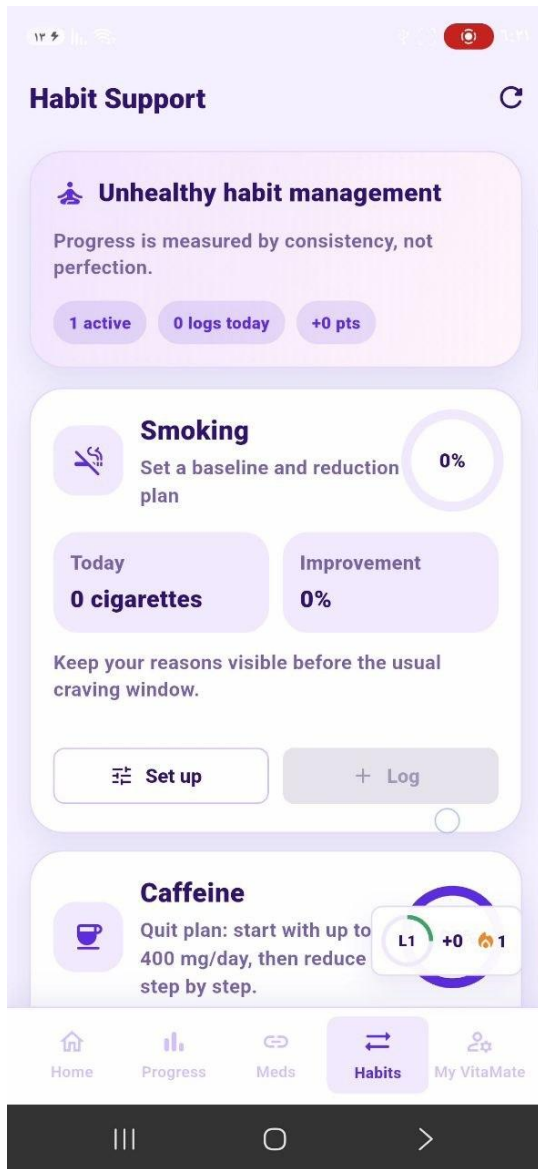


Figure 20: Unhealthy-Habit Management Interface

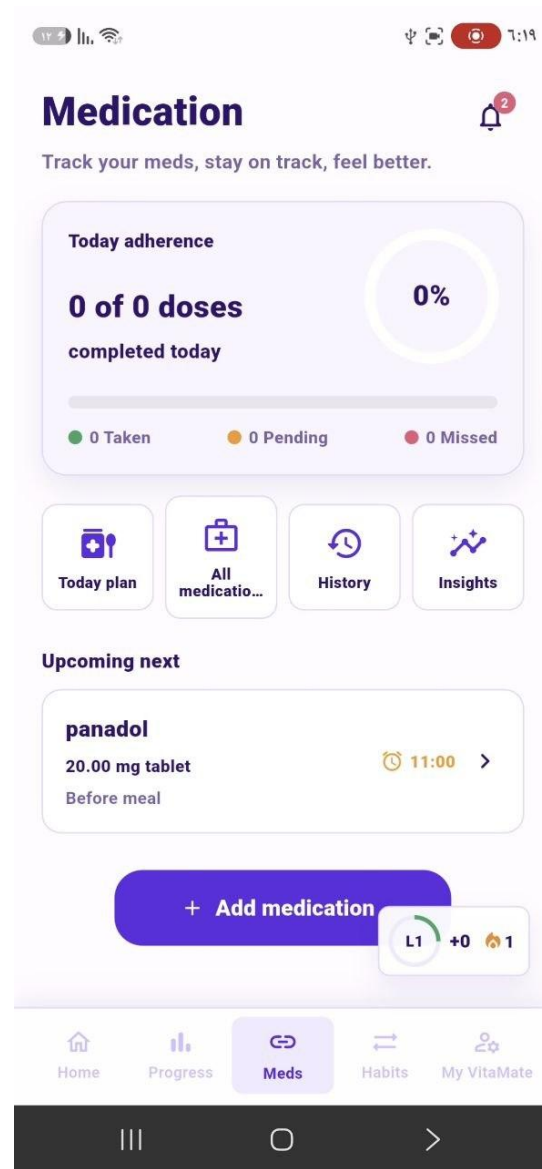


Figure 21: Medication Management Interface

The Home, Nutrition, Activity, and Profile interfaces are shown in Chapter 6 because they directly demonstrate the Senior Project 2 UI/UX refinement. The screens above provide additional operational examples retained in the manual rather than repeated in the implementation chapter.

D.5 Notifications and Motivation

Notification preferences are configured from the application settings. The user can enable supported categories and timing options; local delivery requires Android notification permission. When preferences change, the application synchronizes the current notification plan and replaces

affected local schedules. Motivational points, missions, streaks, achievements, and feedback are updated from verified progress and remain separate from the health score.

D.6 AI-Assisted Meal Logging

Select the AI meal-analysis option, capture or choose a supported meal image, and submit it for analysis. The result is provisional: review the suggested dish/components, correct mappings or quantities, remove incorrect components, and confirm only after the meal is accurate enough for your record. If the image is invalid, the AI service is unavailable, or the result is incomplete, no final meal should be created automatically.

D.7 Troubleshooting

Table 72: Common User Issues and Responses

Issue	Recommended Action
Backend data does not load	Check network/backend availability, then retry without creating duplicate records.
Notifications are not delivered	Verify Android notification permission, enabled category, schedule, and quiet-hour settings; resynchronize if needed.
Step data is unavailable	Verify the required activity/step permission and supported device service.
AI image is rejected	Use a supported image type/quality and retry; invalid images are rejected before finalization.
AI result is uncertain	Edit the dish/components/quantities or reject the result; use manual logging if necessary.
Displayed totals seem stale	Refresh/reopen the related view after saving or editing the source record.

D.8 Frequently Asked Questions (FAQ)

Can VitaMate diagnose a disease or modify treatment? No. VitaMate provides self-monitoring, reminders, constraints, and supportive information; diagnosis and treatment decisions remain outside the application scope.

Are AI meal results saved automatically? No. AI results remain provisional until the authenticated user reviews and confirms the intended food information and quantities.

Why might notifications fail to appear? Local delivery depends on Android notification permission, the enabled category, the synchronized plan, and operating-system scheduling behavior.

Does the submitted release include AI-generated personalized diet recommendations?

No. The second AI recommendation service defined by SRS FR-60-FR-66 was deferred and is future work.

D.9 Support and Contact

The submitted project is an academic prototype and does not provide a production customer-support channel. Technical questions about the academic build should be directed to the project team or the supervising department using the university's approved communication channels.

Appendix E: Technical Evidence and Representative Code

This appendix retains the technical detail intentionally removed from the main implementation chapter. It provides representative structure, configuration, CI, API, and code evidence without reproducing the entire source repository.

E.1 Simplified Project Structure

VitaMate/

vitamate_backend/ — Django settings/routing; core domains/services/repositories; users; gamification; notification_hub; ai_meals; manage.py

vitamate_frontend/ — lib/{auth, core, features, shared}; test/; integration_test/; pubspec.yaml

E.2 Configuration and CI Evidence

Table 73: Key Configuration Files

File / Area	Purpose
requirements / dependency configuration	Backend and AI dependencies
pubspec.yaml / pubspec.lock	Flutter dependencies and resolved package versions
.env / environment configuration	Runtime configuration; secrets excluded from repository
Django migrations	Relational schema evolution
GitHub Actions workflows	Automated quality, test, and artifact jobs
Test seed/reference-data helpers	Stable repeatable database-dependent tests

Table 74: Main CI Checks

CI Check	Purpose
Gitleaks / secret scanning	Detect accidentally committed secrets
Pre-commit / static checks	Catch formatting/static issues according to repository configuration
Backend tests with PostgreSQL	Verify Django/DRF behavior in CI
Flutter analyze / test / coverage	Verify frontend code and automated tests
Android-emulator integration job	Run enabled mobile integration workflow
Artifact upload	Retain logs and coverage outputs

E.3 Representative Data-Access and Business Logic

```
session = (
    MealAnalysisSession.objects
    .filter(user=user, id=analysis_id)
    .prefetch_related(
        "candidates__mapped_food_item__nutrition_facts"
    )
    .first()
)
```

Figure 22: Representative Django ORM and Data-Access Code

```
aggregate_snapshot = {
    snapshot_field: sum(...)
    for snapshot_field in SNAPSHOT_FIELD_MAP.values()
}

meal_log = MealLogRepository.create_for_user(
    user=user,
    meal_type=meal_type,
    **aggregate_snapshot,
)

MealLogComponent.objects.bulk_create(components)
cls._publish_integrations(meal_log)
```

Figure 23: Representative Backend Business-Logic Code

E.4 Representative API Implementation

```
serializer = AnalyzeMealSerializer(data=request.data)
serializer.is_valid(raise_exception=True)

analysis = service.analyze(
    user=request.user,
    analysis_key=analysis_key,
    **serializer.validated_data,
)

return Response(
    {
        "data": MealAnalysisSessionSerializer(
            analysis,
            context={"request": request},
        ).data
    },
    status=status.HTTP_201_CREATED,
)
```

Figure 24: Representative AI-Meal REST Endpoint Implementation

The representative code demonstrates the separation between request handling, user ownership, data access, and service-layer processing. The complete source repository is intentionally not embedded in the report.

E.5 Representative Relational Schema

The table below summarizes the logical relational schema visible in the documented ERD. Logical type categories are used to avoid inventing physical SQL types that are controlled by Django model definitions and migrations. Primary and foreign-key relationships are reported explicitly. The diet-plan relation belongs to the deferred second AI recommendation service and is marked accordingly.

Table 75: Representative Logical Relational Schema

Relation	Primary Key	Main Foreign Keys	Representative Attributes / Logical Types	Final Status
Users	User_ID	-	user_email (text); password_hash (text); created_at (datetime)	Implemented
User_Profile	User_ID	User_ID -> Users	full_name (text); date_of_birth (date); gender (text); height_cm/weight_kg (numeric); activity_level/primary_g oal (text); current_points (numeric)	Implemented
Activity_Logs	Activity_Log_ID	user_id -> Users	activity_kind (text); steps_count/calories_bur ned/duration_minutes (numeric); logged_at (datetime)	Implemented
Sleep_Logs	Sleep_Log_ID	user_id -> Users	sleep_start/sleep_en d (datetime)	Implemented
Goals	Goal_ID	User_ID -> Users; User_Condition_ID -> User_Condition; Nutrient_ID -> Nutrients	goal_type/status (text); target_value (numeric); target_unit (text); start_date (date)	Implemented
Reminders	Reminder_ID	User_ID -> Users; medication_id; habit_id; goal_id	reminder_type (text); scheduled_time (datetime/time); is_active (boolean)	Implemented
User_Condition	user_condition_id	user_id -> Users	condition_type/stat us (text)	Implemented
health_metric_logs	metric_log_id	user_id -> Users; user_condition_id -> User_Condition	metric_type (text); primary_value/secondary _value (numeric); recorded_at (datetime)	Implemented
Medications	medication_id	user_id -> Users; user_condition_id -> User_Condition	medication_type/na me (text); dosage/frequency_type (text)	Implemented
Medication_Logs	medication_log_id	medication_id -> Medications	scheduled_at/taken _at (datetime); dose_status (text)	Implemented

Relation	Primary Key	Main Foreign Keys	Representative Attributes / Logical Types	Final Status
user_habits	habit_id	user_id -> Users	habit_name/target_mode/status (text); daily_limit (numeric)	Implemented
habit_logs	habit_log_id	habit_id -> user_habits	logged_at (datetime); quantity (numeric); adherence_status (text); relapse_flag (boolean)	Implemented
Intake_Items_Catalog	catalog_item_id / intake_item_id	-	name/item_type/default_unit (text); default_amount (numeric)	Implemented
Intake_Items_Nutrients	catalog_item_id + Nutrient_ID	catalog_item_id -> Intake_Items_Catalog; Nutrient_ID -> Nutrients	amount_per_100 (numeric)	Implemented
Nutrients	Nutrient_ID	-	name/nutrient_type/unit (text)	Implemented
Intake_Logs	Intake_log_id	user_id -> Users; catalog_item_id -> Intake_Items_Catalog	consumed_amount (numeric); consumed_unit/intake_type (text)	Implemented
Progress_Reports	report_id	user_id -> Users	module_name/report_type (text); completion_rate/points_earned (numeric)	Implemented
diet_plan_entries	Plan_Entry_ID	User_ID -> Users; intake_item_id -> Intake_Items_Catalog	plan_group_id/day_index (identifier/integer); intake_slot/alternative_text (text); quantity/target_calories (numeric)	Deferred - second AI recommendation service

E.6 Representative API Catalogue

The catalogue records exact paths only where they are documented in the submitted project evidence. Other API groups are described by their routed resource family rather than inventing deployment paths. Full route definitions remain in the Django URL/router configuration.

Table 76: Representative API Interface Catalogue

API / Route	Method	Purpose	Main Input	Main Response	Representative Errors
/api/auth/me/	GET / update operation	Read or update authenticated user/profile data	JWT; profile fields	User/profile representation	401/403 authentication or ownership; 400 validation
/api/dashboard/	GET	Return coordinated daily dashboard state	JWT; optional date/context parameters	Aggregated tracker/progress state	401/403; validation/service errors

API / Route	Method	Purpose	Main Input	Main Response	Representative Errors
/api/history/	GET	Return multi-day progress/history summaries	JWT; date/range parameters where supported	History/read-model response	401/403; validation/service errors
Health tracker route families	GET/POST/PUT/DELETE as applicable	Nutrition, hydration, activity/steps, sleep, medication, conditions, goals, and habits	JWT; resource-specific payloads	Resource or summary JSON	400 validation; 401/403; 404; 5xx on server failure
Motivation route family	GET and event-driven backend updates	Read motivational state/feed and process eligible verified actions	JWT; verified domain event/context	Points/missions/streaks/achievements/feed state	400/401/403; duplicate reward prevented
Notification route family	GET/POST/PUT as applicable	Register device, store preferences, synchronize notification plan	JWT; device/preferences/plan revision	Preferences and current plan/state	400/401/403; permission/device constraints handled client-side
AI meal route family (Django -> FastAPI internally)	POST analyze; review/confirm/finalize operations	Analyze a supported meal image and manage provisional review/finalization	JWT; multipart image; reviewed components/quantities	Provisional analysis session or confirmed meal	400 invalid input; 401/403; 503/internal AI failure; no final meal on failure

Appendix F: Research Ethics and Use of Artificial Intelligence Tools

AI assistance was used as a supporting tool during project documentation and selected explanation/debugging activities. Its use did not replace the students' responsibility for implementation, integration, testing, source verification, or interpretation of results.

Table 77: AI-Assistance Disclosure

Tool	Tasks Supported	Human Review and Control
OpenAI ChatGPT	Brainstorming, language refinement, report restructuring, and selected code/debugging explanations	All suggestions were reviewed, edited, checked against the implementation or project evidence, and accepted only when consistent with the students' work.

No experimental data, test results, references, performance values, or clinical claims were fabricated through AI assistance. Final technical decisions, code changes, test execution, and academic interpretation remained the responsibility of the students. AI-generated wording was treated as draft material and revised to match the verified project scope and evidence.